Personal AI Coding Agent

Complete Tools Catalog with Code

173 tools across 17 categories — production-style Python implementations

agent_tools/ package

Each tool below is a real, runnable function from the `agent_tools` package. Shared safety helpers (workspace validation, bounded subprocess execution, secret redaction, audit logging) live in `agent_tools/core.py` and are shown first.

Core design

The agent works in a loop: understand the goal, inspect the repository, plan, make a small reviewable change, run focused checks, read errors and iterate, then summarize. The model **chooses and explains actions**; a separate tool layer executes them under explicit permissions. The model never gets unrestricted access to the OS, credentials, or network.

> **Implementation note:** this PDF contains working Python code for every tool. Start with the 10-tool minimal set (listed at the end) and expand as reliability improves.

Shared safety core — agent_tools/core.py

```
"""Shared runtime helpers used by every agent tool.

Security baseline implemented here:
  * All file access is confined to the approved workspace root.
  * Commands run with bounded output and timeouts.
  * Every tool action is written to an audit log.
"""
from __future__ import annotations

import json
import os
import re
import subprocess
import sys
import time
from pathlib import Path

WORKSPACE = Path(os.environ.get("AGENT_WORKSPACE", os.getcwd())).resolve()
STATE_DIR = WORKSPACE / ".agent"
AUDIT_LOG = STATE_DIR / "audit.jsonl"
MAX_OUTPUT = 20_000  # characters retained from any command output


class ToolError(RuntimeError):
    """Raised when a tool cannot complete safely or an approval is missing."""


def in_workspace(path: str | Path) -> Path:
    """Resolve a path and reject anything that escapes the approved workspace root."""
    p = Path(path)
    p = (p if p.is_absolute() else WORKSPACE / p).resolve()
    if p != WORKSPACE and WORKSPACE not in p.parents:
        raise ToolError(f"Path escapes approved workspace: {p}")
    return p


def audit(tool: str, args: dict, outcome: str) -> None:
    """Append one JSON line describing a tool action (no secrets)."""
    STATE_DIR.mkdir(parents=True, exist_ok=True)
    record = {"ts": time.time(), "tool": tool, "args": _redact(args), "outcome": outcome}
    with AUDIT_LOG.open("a", encoding="utf-8") as fh:
        fh.write(json.dumps(record, default=str) + "\n")


_SECRET_RE = re.compile(
    r"(?i)(api[_-]?key|secret|token|password|passwd|authorization|credential)"
    r"(\s*[:=]\s*)(\S+)"
)


def _redact(obj):
    """Recursively redact values that look like secrets before logging/returning."""
    if isinstance(obj, dict):
        return {k: ("[REDACTED]" if _SECRET_RE.search(str(k)) else _redact(v)) for k, v in obj.items()}
    if isinstance(obj, (list, tuple)):
        return [_redact(v) for v in obj]
    if isinstance(obj, str):
        return _SECRET_RE.sub(lambda m: m.group(1) + m.group(2) + "[REDACTED]", obj)
    return obj
```

```
def redact_sensitive_output(text: str) -> str:
    """Remove tokens, passwords, and personal data from logs before display."""
    return _SECRET_RE.sub(lambda m: m.group(1) + m.group(2) + "[REDACTED]", text)


def run(argv: list[str], cwd: str | Path | None = None, timeout: int = 120,
        shell: bool = False) -> dict:
    """Run a command with a time limit and bounded output; never raises on failure."""
    cwd = str(cwd or WORKSPACE)
    try:
        proc = subprocess.run(
            argv, cwd=cwd, shell=shell, timeout=timeout,
            capture_output=True, text=True, errors="replace",
        )
        out = redact_sensitive_output(proc.stdout or "")[:MAX_OUTPUT]
        err = redact_sensitive_output(proc.stderr or "")[:MAX_OUTPUT]
        return {"argv": argv, "exit_code": proc.returncode, "stdout": out, "stderr": err}
    except subprocess.TimeoutExpired:
        return {"argv": argv, "exit_code": -1, "stdout": "", "stderr": f"Timed out after {timeout}s"}
    except FileNotFoundError:
        return {"argv": argv, "exit_code": -1, "stdout": "", "stderr": f"Executable not found: {argv[0]}"}


def which(cmd: str) -> str | None:
    """Locate an executable on PATH."""
    import shutil
    return shutil.which(cmd)


def read_json(path: Path, default=None):
    if not path.exists():
        return default
    return json.loads(path.read_text(encoding="utf-8"))


def write_json(path: Path, data) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(data, indent=2, default=str), encoding="utf-8")


def model_call(prompt: str, system: str | None = None, model: str | None = None) -> str:
    """Send a structured prompt to the configured OpenAI-compatible model endpoint.

    Configure via env: LLM_BASE_URL, LLM_API_KEY, LLM_MODEL. Raise ToolError if unset.
    """
    import urllib.request

    base = os.environ.get("LLM_BASE_URL")
    key = os.environ.get("LLM_API_KEY")
    name = model or os.environ.get("LLM_MODEL", "gpt-4o-mini")
    if not base or not key:
        raise ToolError("No model configured. Set LLM_BASE_URL, LLM_API_KEY, LLM_MODEL.")

    messages = []
    if system:
        messages.append({"role": "system", "content": system})
    messages.append({"role": "user", "content": prompt})
    payload = json.dumps({"model": name, "messages": messages, "temperature": 0.1}).encode()
    req = urllib.request.Request(
        base.rstrip("/") + "/chat/completions", data=payload,
        headers={"Content-Type": "application/json", "Authorization": f"Bearer {key}"},
    )
```

```
    try:
        with urllib.request.urlopen(req, timeout=120) as resp:
            body = json.loads(resp.read().decode())
    except Exception as exc:  # noqa: BLE001 - surface any provider failure to the agent loop
        raise ToolError(f"Model call failed: {exc}") from exc
    return body["choices"][0]["message"]["content"]


def cli_main(registry: dict) -> None:
    """Tiny CLI: `python -m agent_tools.<module> <tool> '<json args>'`."""
    if len(sys.argv) < 2:
        print("Tools:", ", ".join(sorted(registry)))
        sys.exit(0)
    tool = sys.argv[1]
    args = json.loads(sys.argv[2]) if len(sys.argv) > 2 else {}
    result = registry[tool](**args)
    print(json.dumps(result, indent=2, default=str))
```

2.1 File-system tools

Tool	Purpose & constraints
`list_files`	List files and folders in a project — exclude .git, caches, build output, secrets by default.
`tree_project`	Show a concise directory tree — limit depth and entry count.
`read_file`	Read a text file — enforce project-root boundaries and size limits.
`read_file_range`	Read selected lines from a large file — return line numbers.
`search_files`	Search filenames and file contents — ignore generated folders.
`file_metadata`	Inspect size, type, and modification time.
`create_file`	Create a new file — require a valid path inside the workspace.
`write_file`	Write or replace a file — show a diff for approval.
`apply_patch`	Apply a structured edit — fail safely if the target no longer matches.
`move_file`	Rename or move a file — block moves outside the workspace.
`copy_file`	Copy project files — preserve encoding and permissions.
`delete_file`	Delete a file — require explicit approval; prefer backup.
`find_references`	Find usages of a symbol or file — word-boundary matching.

list_files()

```
def list_files(path: str = ".", pattern: str = "*") -> list[str]:
    """List files and folders in a project, excluding caches/build/secrets by default."""
    root = in_workspace(path)
    out = []
    for dirpath, dirnames, filenames in os.walk(root):
        dirnames[:] = [d for d in dirnames if d not in EXCLUDE_DIRS and not d.startswith(".")]
        for name in filenames:
            if fnmatch.fnmatch(name, pattern) and not name.startswith("."):
                out.append(str(Path(dirpath, name).relative_to(root)))
    audit("list_files", {"path": path, "pattern": pattern}, f"{len(out)} entries")
    return out
```

tree_project()

```
def tree_project(path: str = ".", max_depth: int = 3, max_entries: int = 300) -> str:
    """Show a concise directory tree, limited by depth and entry count."""
    root = in_workspace(path)
    lines, count = [], 0

    def walk(dirpath: Path, prefix: str, depth: int):
        nonlocal count
        if depth > max_depth or count >= max_entries:
            return
        entries = sorted([e for e in dirpath.iterdir()
                          if e.name not in EXCLUDE_DIRS and not e.name.startswith(".")],
                         key=lambda e: (e.is_file(), e.name.lower()))
        for entry in entries:
            if count >= max_entries:
                return
            count += 1
            lines.append(prefix + entry.name + ("/" if entry.is_dir() else ""))
            if entry.is_dir():
                walk(entry, prefix + "  ", depth + 1)

    walk(root, "", 1)
    audit("tree_project", {"path": path}, f"{count} entries")
    return "\n".join(lines)
```

read_file()

```
def read_file(path: str, max_bytes: int = 200_000) -> str:
    """Read a text file inside the workspace with a size limit."""
    p = in_workspace(path)
    if not p.is_file():
        raise ToolError(f"Not a file: {p}")
    if p.stat().st_size > max_bytes:
        raise ToolError(f"File too large ({p.stat().st_size} bytes); use read_file_range.")
    text = p.read_text(encoding="utf-8", errors="replace")
    audit("read_file", {"path": path}, f"{len(text)} chars")
    return text
```

read_file_range()

```
def read_file_range(path: str, start_line: int = 1, end_line: int | None = None) -> str:
    """Read selected lines from a large file; returns line-numbered text."""
    p = in_workspace(path)
    lines = p.read_text(encoding="utf-8", errors="replace").splitlines()
    end_line = end_line or min(len(lines), start_line + 500)
    window = lines[start_line - 1:end_line]
    audit("read_file_range", {"path": path, "start": start_line, "end": end_line}, "ok")
    return "\n".join(f"{i}: {l}" for i, l in enumerate(window, start=start_line))
```

search_files()

```
def search_files(query: str, path: str = ".", regex: bool = False,
                 max_results: int = 100) -> list[str]:
    """Search file contents (and names) with ripgrep when available, else Python fallback."""
    from .core import run, which
    root = in_workspace(path)
    if which("rg"):
        flags = ["-n", "--no-heading"] + (["-e"] if regex else ["-F"])
        res = run(["rg", *flags, query, str(root), "--glob", "!node_modules",
                   "--glob", "!.git", "--max-count", "1"])
        matches = res["stdout"].splitlines()[:max_results]
    else:
        pat = re.compile(query if regex else re.escape(query))
        matches = []
        for rel in list_files(str(root)):
            p = root / rel
            try:
                for n, line in enumerate(p.read_text(encoding="utf-8", errors="replace").splitlines(), 1):
                    if pat.search(line):
                        matches.append(f"{rel}:{n}: {line.strip()[:160]}")
                        if len(matches) >= max_results:
                            break
            except OSError:
                continue
    audit("search_files", {"query": query}, f"{len(matches)} matches")
    return matches
```

file_metadata()

```
def file_metadata(path: str) -> dict:
    """Inspect size, type, and modification time of a workspace file."""
    p = in_workspace(path)
    st = p.stat()
    audit("file_metadata", {"path": path}, "ok")
    return {"path": str(p), "size": st.st_size, "is_dir": p.is_dir(),
            "suffix": p.suffix, "modified": st.st_mtime}
```

create_file()

```
def create_file(path: str, content: str = "") -> str:
    """Create a new file (fail if it already exists) inside the workspace."""
    p = in_workspace(path)
    if p.exists():
        raise ToolError(f"File already exists: {p}")
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    audit("create_file", {"path": path}, "created")
    return f"Created {p}"
```

write_file()

```
def write_file(path: str, content: str, overwrite: bool = True) -> dict:
    """Write or replace a file; returns the diff preview for approval flows."""
    import difflib
    p = in_workspace(path)
    old = p.read_text(encoding="utf-8") if p.exists() else ""
    if p.exists() and not overwrite:
        raise ToolError(f"Refusing to overwrite {p}")
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    diff = "\n".join(difflib.unified_diff(old.splitlines(), content.splitlines(),
                                          str(p), str(p), lineterm=""))
    audit("write_file", {"path": path}, "written")
    return {"path": str(p), "diff": diff[:8000]}
```

apply_patch()

```
def apply_patch(path: str, old: str, new: str, count: int = 1) -> dict:
    """Apply a structured edit; fails safely if the target text no longer matches."""
    import difflib
    p = in_workspace(path)
    text = p.read_text(encoding="utf-8")
    if old not in text:
        raise ToolError(f"Patch target not found in {p}; file may have changed.")
    updated = text.replace(old, new, count)
    diff = "\n".join(difflib.unified_diff(text.splitlines(), updated.splitlines(),
                                          str(p), str(p), lineterm=""))
    p.write_text(updated, encoding="utf-8")
    audit("apply_patch", {"path": path}, "applied")
    return {"path": str(p), "diff": diff[:8000]}
```

move_file()

```
def move_file(src: str, dst: str) -> str:
    """Rename or move a file; block moves outside the workspace by default."""
    s, d = in_workspace(src), in_workspace(dst)
    d.parent.mkdir(parents=True, exist_ok=True)
    shutil.move(str(s), str(d))
    audit("move_file", {"src": src, "dst": dst}, "moved")
    return f"Moved {s} -> {d}"
```

copy_file()

```
def copy_file(src: str, dst: str) -> str:
    """Copy project files preserving encoding (binary-safe copy)."""
    s, d = in_workspace(src), in_workspace(dst)
    d.parent.mkdir(parents=True, exist_ok=True)
    shutil.copy2(s, d)
    audit("copy_file", {"src": src, "dst": dst}, "copied")
    return f"Copied {s} -> {d}"
```

delete_file()

```
def delete_file(path: str, approved: bool = False) -> str:
    """Delete a file only with explicit approval; prefer move-to-backup otherwise."""
    p = in_workspace(path)
    if not approved:
        backup = p.with_suffix(p.suffix + ".bak")
        shutil.move(str(p), str(backup))
        audit("delete_file", {"path": path}, f"moved to {backup}")
        return f"Not approved; moved to {backup}"
    if p.is_dir():
        shutil.rmtree(p)
    else:
        p.unlink()
    audit("delete_file", {"path": path}, "deleted")
    return f"Deleted {p}"
```

find_references()

```
def find_references(symbol: str, path: str = ".", max_results: int = 100) -> list[str]:
    """Find usages of a symbol or file with word-boundary matching."""
    return search_files(rf"\b{re.escape(symbol)}\b", path, regex=True, max_results=max_results)
```

2.2 Terminal and process tools

Tool	Purpose & constraints
`run_command`	Run an approved command — allowlist, timeout, bounded output.
`run_script`	Execute a project script — explicit interpreter and working directory.
`start_process`	Start a long-running dev server/watcher — return PID and logs.
`read_process_output`	Read stdout/stderr from an agent-owned process — bounded.
`stop_process`	Stop an agent-owned process — refuse unowned processes.
`check_command_exists`	Check whether a tool is installed — version + install hint.
`environment_info`	Inspect OS, runtime, tool versions — redact secrets.
`set_working_directory`	Choose the active directory — restrict to workspace roots.
`run_with_timeout`	Run a command with a hard time limit — kill children safely.

run_command()

```
def run_command(argv: list[str], timeout: int = 120, cwd: str | None = None,
                require_approval: bool = True) -> dict:
    """Run an approved command in the project with timeout and bounded output."""
    if not isinstance(argv, list):
        raise ToolError("run_command expects an argument array, not a shell string.")
    if not verify_command_allowlist(argv) and require_approval:
        audit("run_command", {"argv": argv}, "denied: not allowlisted")
        raise ToolError(f"'{argv[0]}' is not on the allowlist; requires explicit approval.")
    result = run(argv, cwd=cwd, timeout=timeout)
    audit("run_command", {"argv": argv}, f"exit {result['exit_code']}")
    return result
```

run_script()

```
def run_script(script: str, interpreter: str = "python", cwd: str | None = None) -> dict:
    """Execute a project script with an explicit interpreter and working directory."""
    p = in_workspace(script)
    interp = which(interpreter) or interpreter
    audit("run_script", {"script": script, "interpreter": interpreter}, "started")
    return run([interp, str(p)], cwd=cwd or str(p.parent))
```

start_process()

```
def start_process(argv: list[str], name: str = "dev", log_dir: str = ".agent/logs") -> dict:
    """Start a long-running dev server/watcher; returns a process id and log path."""
    log_path = in_workspace(log_dir) / f"{name}.log"
    log_path.parent.mkdir(parents=True, exist_ok=True)
    fh = open(log_path, "ab")
    proc = subprocess.Popen(argv, cwd=str(in_workspace(".")), stdout=fh, stderr=fh)
    _PROCESSSES[name] = {"proc": proc, "log": log_path, "started": time.time()}
    audit("start_process", {"argv": argv, "name": name}, f"pid {proc.pid}")
    return {"name": name, "pid": proc.pid, "log": str(log_path)}
```

read_process_output()

```
def read_process_output(name: str, lines: int = 100) -> dict:
    """Read recent stdout/stderr captured from an agent-owned process."""
    entry = _PROCESSSES.get(name)
    if not entry:
        raise ToolError(f"Unknown process: {name}")
    log: Path = entry["log"]
    tail = log.read_text(encoding="utf-8", errors="replace").splitlines()[-lines:]
    proc: subprocess.Popen = entry["proc"]
    audit("read_process_output", {"name": name}, "ok")
    return {"name": name, "running": proc.poll() is None, "output": "\n".join(tail)[-MAX_OUTPUT:]}
```

stop_process()

```
def stop_process(name: str) -> dict:
    """Stop an agent-owned process; refuse to stop processes the agent did not start."""
    entry = _PROCESSSES.pop(name, None)
    if not entry:
        raise ToolError(f"Process '{name}' was not started by this agent; refusing to stop it.")
    proc: subprocess.Popen = entry["proc"]
    proc.terminate()
    try:
        proc.wait(timeout=10)
    except subprocess.TimeoutExpired:
        proc.kill()
    audit("stop_process", {"name": name}, "stopped")
    return {"name": name, "stopped": True, "exit_code": proc.returncode}
```

check_command_exists()

```
def check_command_exists(cmd: str) -> dict:
    """Check whether a tool is installed; report version and install guidance."""
    path = which(cmd)
    version = None
    if path:
        res = run([cmd, "--version"], timeout=20)
        version = (res["stdout"] or res["stderr"]).splitlines()[0] if (res["stdout"] or res["stderr"]) else
None
    audit("check_command_exists", {"cmd": cmd}, "found" if path else "missing")
    return {"command": cmd, "installed": bool(path), "path": path, "version": version,
            "install_hint": None if path else f"Install '{cmd}' with your OS package manager."}
```

environment_info()

```
def environment_info() -> dict:
    """Inspect OS, runtime, and tool versions; secrets and env vars are never returned."""
    import platform
    info = {
        "os": platform.system(),
        "os_release": platform.release(),
        "python": platform.python_version(),
        "cwd": os.getcwd(),
        "shell": os.environ.get("SHELL") or os.environ.get("COMSPEC"),
    }
    for tool in ("git", "node", "npm", "rg", "docker"):
        path = which(tool)
        if path:
            info[tool] = (run([tool, "--version"], timeout=15)["stdout"] or "").strip().splitlines()[:1]
    audit("environment_info", {}, "ok")
    return info
```

set_working_directory()

```
def set_working_directory(path: str) -> str:
    """Choose the active project directory, restricted to approved workspace roots."""
    p = in_workspace(path)
    if not p.is_dir():
        raise ToolError(f"Not a directory: {p}")
    os.chdir(p)
    audit("set_working_directory", {"path": path}, "changed")
    return f"cwd -> {p}"
```

run_with_timeout()

```
def run_with_timeout(argv: list[str], timeout: int = 60, cwd: str | None = None) -> dict:
    """Run a command with a hard time limit, terminating child processes safely."""
    result = run(argv, cwd=cwd, timeout=timeout)
    audit("run_with_timeout", {"argv": argv, "timeout": timeout}, f"exit {result['exit_code']}")
    return result
```

2.3 Code creation and editing

Tool	Purpose & constraints
`generate_code`	Produce code for a specific task or file.
`edit_code`	Modify an existing function, class, or component.
`apply_diff`	Apply a reviewed patch to one or more files.
`refactor_symbol`	Rename/restructure a symbol while preserving references.
`extract_function`	Move repeated logic into a function or method.
`add_type_annotations`	Add or improve types where supported.
`add_error_handling`	Add appropriate validation and error handling.
`remove_dead_code`	Identify and remove unused code after verification.
`format_file`	Run the project's formatter on selected files.
`organize_imports`	Sort and remove unused imports.
`generate_config`	Create configuration files for a framework or tool.
`generate_scaffold`	Create a project skeleton from a selected template.

generate_code()

```
def generate_code(instruction: str, path: str | None = None) -> dict:
    """Produce code for a specific task or file via the configured model."""
    prompt = f"Write code for the following task. Output only code:\n{instruction}"
    code = model_call(prompt, system="You are a careful coding assistant. Preserve project conventions.")
    audit("generate_code", {"instruction": instruction[:200], "path": path}, "generated")
    if path:
        result = write_file(path, code if code.endswith("\n") else code + "\n")
        return {"path": path, "diff": result["diff"]}
    return {"code": code}
```

edit_code()

```
def edit_code(path: str, old: str, new: str) -> dict:
    """Modify an existing function, class, or component with an exact-match edit."""
    return apply_patch(path, old, new)
```

apply_diff()

```
def apply_diff(path: str, old: str, new: str) -> dict:
    """Apply a reviewed patch to one or more files (fails if the target changed)."""
    return apply_patch(path, old, new)
```

refactor_symbol()

```
def refactor_symbol(old_name: str, new_name: str, path: str = ".") -> dict:
    """Rename a symbol everywhere while preserving references (word-boundary safe)."""
    from .filesystem import list_files
    pattern = re.compile(rf"\b{re.escape(old_name)}\b")
    changed = []
    for rel in list_files(path):
        p = in_workspace(rel if path == "." else f"{path}/{rel}")
        if p.suffix not in {".py", ".js", ".ts", ".jsx", ".tsx", ".dart", ".go", ".rs", ".java"}:
            continue
        text = p.read_text(encoding="utf-8", errors="replace")
        if pattern.search(text):
            p.write_text(pattern.sub(new_name, text), encoding="utf-8")
            changed.append(str(p))
    audit("refactor_symbol", {"old": old_name, "new": new_name}, f"{len(changed)} files")
    return {"renamed": new_name, "files_changed": changed}
```

extract_function()

```
def extract_function(path: str, start_line: int, end_line: int, name: str) -> dict:
    """Move repeated logic into a new function/method (model-assisted, then verify)."""
    p = in_workspace(path)
    lines = p.read_text(encoding="utf-8").splitlines(keepends=True)
    body = "".join(lines[start_line - 1:end_line])
    prompt = (f"Extract the following lines into a function named {name} in {path}. "
              f"Return the new function plus the replacement call site, separated by "
              f"'---REPLACE---'.\n{body}")
    return {"result": model_call(prompt), "note": "Apply with apply_patch after review."}
```

add_type_annotations()

```
def add_type_annotations(path: str) -> dict:
    """Add or improve types using the project's type checker where supported."""
    p = in_workspace(path)
    if which("mypy"):
        before = run(["mypy", "--follow-imports=skip", str(p)], timeout=120)
        prompt = (f"Add type annotations to this file to fix mypy errors. Return only the "
                  f"updated full file.\n```\n{p.read_text(encoding='utf-8')[:12000]}\n```")
        updated = model_call(prompt)
        p.write_text(updated, encoding="utf-8")
        after = run(["mypy", "--follow-imports=skip", str(p)], timeout=120)
        audit("add_type_annotations", {"path": path}, f"mypy errors {len(before['stdout'].splitlines())} ->
{len(after['stdout'].splitlines())}")
        return {"before_errors": len(before["stdout"].splitlines()),
                "after_errors": len(after["stdout"].splitlines())}
    raise ToolError("mypy not installed; cannot verify type annotations.")
```

add_error_handling()

```
def add_error_handling(path: str, focus: str = "") -> dict:
    """Add appropriate validation and error handling via the model, then diff."""
    p = in_workspace(path)
    prompt = (f"Add appropriate validation and error handling to this file. "
              f"Focus: {focus or 'all risky operations'}. Return only the updated file.\n"
              f"```\n{p.read_text(encoding='utf-8')[:12000]}\n```")
    updated = model_call(prompt)
    result = write_file(path, updated if updated.endswith("\n") else updated + "\n")
    return {"path": path, "diff": result["diff"]}
```

remove_dead_code()

```
def remove_dead_code(path: str, symbol: str) -> dict:
    """Identify and remove unused code only after verifying no references remain."""
    from .filesystem import find_references
    refs = [r for r in find_references(symbol, path) if symbol not in r.split(":")[-1] or True]
    own = in_workspace(path)
    refs = [r for r in refs if not r.startswith(str(own))]
    if refs:
        raise ToolError(f"Symbol '{symbol}' still referenced in {len(refs)} places; not removing.")
    return edit_code(path, symbol, "")
```

format_file()

```
def format_file(path: str) -> dict:
    """Run the project's formatter on selected files."""
    p = in_workspace(path)
    for cmd in (["ruff", "format", str(p)], ["black", "-q", str(p)],
                ["prettier", "--write", str(p)], ["dart", "format", str(p)]):
        if which(cmd[0]):
            res = run(cmd, timeout=120)
            audit("format_file", {"path": path, "formatter": cmd[0]}, f"exit {res['exit_code']}")
            return {"formatter": cmd[0], "exit_code": res["exit_code"], "output": res["stdout"]}
    raise ToolError("No formatter found (tried ruff, black, prettier, dart).")
```

organize_imports()

```
def organize_imports(path: str) -> dict:
    """Sort and remove unused imports with the project's tooling."""
    p = in_workspace(path)
    if which("ruff"):
        res = run(["ruff", "check", "--select", "I,F401", "--fix", str(p)], timeout=60)
        return {"tool": "ruff", "exit_code": res["exit_code"], "output": res["stdout"]}
    if which("autoflake"):
        res = run(["autoflake", "--in-place", "--remove-unused-imports", str(p)], timeout=60)
        return {"tool": "autoflake", "exit_code": res["exit_code"], "output": res["stdout"]}
    raise ToolError("No import organizer found (tried ruff, autoflake).")
```

generate_config()

```
def generate_config(tool: str, options: dict | None = None) -> dict:
    """Create configuration files for a chosen framework or tool via the model."""
    prompt = (f"Generate a {tool} configuration file for this project. "
              f"Requirements: {options or 'follow current best practices'}. "
              f"Return only the file content.")
    return {"tool": tool, "config": model_call(prompt)}
```

generate_scaffold()

```
def generate_scaffold(template: str, target_dir: str) -> dict:
    """Create a project skeleton using the ecosystem's own scaffolder."""
    commands = {
        "vite": ["npm", "create", "vite@latest", target_dir],
        "next": ["npx", "create-next-app@latest", target_dir],
        "flutter": ["flutter", "create", target_dir],
        "cargolib": ["cargo", "new", "lib", target_dir],
    }
    if template not in commands:
        raise ToolError(f"Unknown template '{template}'. Known: {sorted(commands)}")
    res = run(commands[template], timeout=300)
    audit("generate_scaffold", {"template": template}, f"exit {res['exit_code']}")
    return {"exit_code": res["exit_code"], "output": res["stdout"] + res["stderr"]}
```

2.4 Debugging and diagnostics

Tool	Purpose & constraints
`parse_error`	Extract the actionable portion of an error.
`explain_traceback`	Identify likely failure points in a stack trace.
`locate_error_line`	Map an error to a file and line number.
`inspect_logs`	Search application and build logs for failures.
`reproduce_failure`	Run a minimal, bounded reproduction command.
`inspect_config`	Check configuration for inconsistencies.
`compare_environment`	Compare runtime/dependency versions across environments.
`diagnose_dependency_conflict`	Investigate incompatible package constraints.
`suggest_fix`	Propose fixes ranked by confidence and risk.
`verify_fix`	Rerun the reproduction or test after a change.

parse_error()

```
def parse_error(text: str) -> dict:
    """Extract the actionable portion of a compiler or runtime error."""
    lines = text.strip().splitlines()
    error_lines = [l for l in lines if re.search(r'(?i)\b(error|exception|failed|fatal)\b', l)]
    locations = [{"file": m["file"], "line": int(m["line"])}
                 for m in (_PY_TRACE_LINE.search(l) or _FILE_LINE.search(l) for l in lines) if m]
    audit("parse_error", {"chars": len(text)}, f"{len(error_lines)} error lines")
    return {"summary": error_lines[-1][:300] if error_lines else lines[-1][:300] if lines else "",
            "error_lines": error_lines[-10:], "locations": locations[:10]}
```

explain_traceback()

```
def explain_traceback(text: str) -> dict:
    """Identify likely failure points in a stack trace (innermost frame first)."""
    frames = [{"file": m["file"], "line": int(m["line"])}
              for m in (_PY_TRACE_LINE.search(l) for l in text.splitlines()) if m]
    if not frames:
        frames = [{"file": m["file"], "line": int(m["line"])}
                  for m in (_FILE_LINE.search(l) for l in text.splitlines()) if m]
    if not frames:
        raise ToolError("No recognizable stack frames in the provided text.")
    audit("explain_traceback", {}, f"{len(frames)} frames")
    return {"likely_failure_point": frames[-1], "frames": frames}
```

locate_error_line()

```
def locate_error_line(text: str) -> list[dict]:
    """Map an error to a file and line number, opening the surrounding context."""
    parsed = parse_error(text)
    out = []
    for loc in parsed["locations"]:
        try:
            p = in_workspace(loc["file"])
            lines = p.read_text(encoding="utf-8", errors="replace").splitlines()
            start, end = max(0, loc["line"] - 4), min(len(lines), loc["line"] + 3)
            context = "\n".join(f"{i+1}: {l}" for i, l in enumerate(lines[start:end], start=start))
        except (ToolError, OSError):
            context = None
        out.append({**loc, "context": context})
    return out
```

inspect_logs()

```
def inspect_logs(path: str, pattern: str = r"(?i)(error|exception|fail)", limit: int = 50) -> list[str]:
    """Search application and build logs for failures."""
    p = in_workspace(path)
    rx = re.compile(pattern)
    hits = [f"{i}: {l}" for i, l in enumerate(p.read_text(encoding="utf-8", errors="replace").splitlines(),
1)
            if rx.search(l)][:limit]
    audit("inspect_logs", {"path": path, "pattern": pattern}, f"{len(hits)} hits")
    return hits
```

reproduce_failure()

```
def reproduce_failure(command: list[str], timeout: int = 120) -> dict:
    """Run a minimal, bounded reproduction command and capture the failure."""
    res = run(command, timeout=timeout)
    audit("reproduce_failure", {"command": command}, f"exit {res['exit_code']}")
    return {"reproduced": res["exit_code"] != 0, **res}
```

inspect_config()

```
def inspect_config(path: str) -> dict:
    """Check relevant configuration for inconsistencies (syntax and key sanity)."""
    p = in_workspace(path)
    text = p.read_text(encoding="utf-8", errors="replace")
    issues = []
    if p.suffix == ".json":
        import json
        try:
            json.loads(text)
        except ValueError as exc:
            issues.append(f"Invalid JSON: {exc}")
    for key in ("TODO", "FIXME", "changeme", "your-key-here", "xxx"):
        if key.lower() in text.lower():
            issues.append(f"Placeholder value '{key}' present")
    audit("inspect_config", {"path": path}, f"{len(issues)} issues")
    return {"path": str(p), "issues": issues, "size": len(text)}
```

compare_environment()

```
def compare_environment(current: dict, expected: dict) -> dict:
    """Compare runtime and dependency versions across environments."""
    keys = set(current) | set(expected)
    diffs = {k: {"current": current.get(k), "expected": expected.get(k)}
             for k in sorted(keys) if current.get(k) != expected.get(k)}
    audit("compare_environment", {}, f"{len(diffs)} differences")
    return {"differences": diffs, "match": not diffs}
```

diagnose_dependency_conflict()

```
def diagnose_dependency_conflict(error_text: str) -> dict:
    """Investigate incompatible package constraints from resolver output."""
    m = re.search(r"(?i)(conflict|incompatible|depends on|requires)\s+(.+)", error_text)
    detail = m.group(0) if m else error_text[-500:]
    try:
        analysis = model_call(
            "Explain this dependency conflict and propose the safest resolution "
            f"(prefer minimal version changes):\n{error_text[-2000:]}",
            system="You are a build engineer. Be concise and concrete.")
    except ToolError as exc:
        analysis = f"Model unavailable ({exc}); manual review needed for: {detail}"
    audit("diagnose_dependency_conflict", {}, "analyzed")
    return {"detail": detail, "suggestion": analysis}
```

suggest_fix()

```
def suggest_fix(error_text: str, context: str = "") -> list[dict]:
    """Propose fixes ranked by confidence and risk via the configured model."""
    prompt = (f"Given this error and context, propose up to 3 fixes ranked by confidence. "
              f"Format each as JSON with keys: fix, confidence (0-1), risk (low/med/high).\n"
              f"ERROR:\n{error_text[-2000:]}\nCONTEXT:\n{context[:3000]}")
    raw = model_call(prompt, system="Respond with a JSON array only.")
    import json
    m = re.search(r"\[.*\]", raw, re.S)
    fixes = json.loads(m.group(0)) if m else [{"fix": raw, "confidence": 0.5, "risk": "med"}]
    audit("suggest_fix", {"chars": len(error_text)}, f"{len(fixes)} suggestions")
    return fixes
```

verify_fix()

```
def verify_fix(command: list[str], expected_zero: bool = True, timeout: int = 180) -> dict:
    """Rerun the reproduction or relevant test after a change; the only proof a fix worked."""
    res = run(command, timeout=timeout)
    passed = (res["exit_code"] == 0) == expected_zero
    audit("verify_fix", {"command": command}, "PASS" if passed else "FAIL")
    return {"verified": passed, **res}
```

2.5 Testing and quality

Tool	Purpose & constraints
`run_unit_tests`	Run unit tests for a selected package or file.
`run_integration_tests`	Test interactions between components.
`run_e2e_tests`	Run end-to-end tests where configured.
`run_linter`	Detect style and likely correctness problems.
`run_type_checker`	Check static types.
`run_formatter_check`	Verify formatting without changing files.
`measure_coverage`	Report test coverage if tooling supports it.
`generate_test_cases`	Draft tests for specified behavior.
`run_regression_tests`	Verify that an existing bug remains fixed.
`validate_build`	Build the project or target package.
`summarize_test_results`	Summarize passed/failed/skipped/blocked checks.

run_unit_tests()

```
def run_unit_tests(target: str | None = None) -> dict:
    """Run unit tests for a selected package or file (smallest scope first)."""
    cmd = _detect(["pyproject.toml", "package.json", "Cargo.toml", "go.mod", "pubspec.yaml"])
    if target:
        cmd = cmd + [target]
    return _run_suite(cmd, "unit")
```

run_integration_tests()

```
def run_integration_tests(path: str = "tests/integration") -> dict:
    """Test interactions between components."""
    cmd = _detect([])
    if cmd[0] == "python" or "pytest" in cmd:
        cmd = cmd + [path, "-m", "integration"]
    return _run_suite(cmd, "integration")
```

run_e2e_tests()

```
def run_e2e_tests(path: str = "e2e") -> dict:
    """Run end-to-end tests where configured."""
    root = in_workspace(".")
    if (root / "playwright.config.ts").exists():
        return _run_suite(["npx", "playwright", "test"], "e2e")
    return _run_suite(_detect([]) + [path], "e2e")
```

run_linter()

```
def run_linter(path: str = ".") -> dict:
    """Detect style and likely correctness problems."""
    for cmd in (["ruff", "check", path], ["eslint", path], ["dart", "analyze", path]):
        if which(cmd[0]):
            res = run(cmd, timeout=300)
            audit("run_linter", {"command": cmd}, f"exit {res['exit_code']}")
            return {"tool": cmd[0], "exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])
[-12000:]}
    raise ToolError("No linter found (tried ruff, eslint, dart analyze).")
```

run_type_checker()

```
def run_type_checker(path: str = ".") -> dict:
    """Check static types."""
    for cmd in (["mypy", path], ["npx", "tsc", "--noEmit"], ["pyright", path]):
        if which(cmd[0]):
            res = run(cmd, timeout=600)
            audit("run_type_checker", {"command": cmd}, f"exit {res['exit_code']}")
            return {"tool": cmd[0], "exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])
[-12000:]}
    raise ToolError("No type checker found (tried mypy, tsc, pyright).")
```

run_formatter_check()

```
def run_formatter_check(path: str = ".") -> dict:
    """Verify formatting without changing files."""
    for cmd in (["ruff", "format", "--check", path], ["black", "--check", path],
                ["prettier", "--check", path]):
        if which(cmd[0]):
            res = run(cmd, timeout=300)
            return {"tool": cmd[0], "clean": res["exit_code"] == 0, "output": (res["stdout"] +
res["stderr"])[-8000:]}
    raise ToolError("No formatter check available (tried ruff, black, prettier).")
```

measure_coverage()

```
def measure_coverage(target: str | None = None) -> dict:
    """Report test coverage if tooling supports it."""
    if which("coverage"):
        cmd = ["python", "-m", "coverage", "run", "-m", "pytest", "-q"] + ([target] if target else [])
        run(cmd, timeout=600)
        res = run(["python", "-m", "coverage", "report"], timeout=120)
        audit("measure_coverage", {}, f"exit {res['exit_code']}")
        return {"exit_code": res["exit_code"], "report": res["stdout"][-8000:]}
    res = run(["npm", "test", "--", "--coverage", "--watchAll=false"], timeout=600)
    return {"exit_code": res["exit_code"], "report": (res["stdout"] + res["stderr"])[-8000:]}
```

generate_test_cases()

```
def generate_test_cases(description: str, language: str = "python") -> str:
    """Draft tests for specified behavior via the configured model."""
    prompt = (f"Write unit tests for: {description}\nLanguage/framework: {language}. "
              f"Cover happy path, edge cases, and failure cases. Output only test code.")
    tests = model_call(prompt, system="You write thorough, minimal, correct tests.")
    audit("generate_test_cases", {"description": description[:200]}, "generated")
    return tests
```

run_regression_tests()

```
def run_regression_tests(bug_description: str) -> dict:
    """Verify that an existing bug remains fixed by running related tests."""
    tests = generate_test_cases(f"Regression test proving this bug stays fixed: {bug_description}")
    root = in_workspace(".")
    tf = root / ".agent" / "regression_test.py"
    tf.parent.mkdir(parents=True, exist_ok=True)
    tf.write_text(tests, encoding="utf-8")
    res = run(["python", "-m", "pytest", "-q", str(tf)], timeout=300)
    audit("run_regression_tests", {"bug": bug_description[:200]}, f"exit {res['exit_code']}")
    return {"exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])[-8000:]}
```

validate_build()

```
def validate_build() -> dict:
    """Build the project or target package and report success honestly."""
    root = in_workspace(".")
    if (root / "package.json").exists():
        return _run_suite(["npm", "run", "build"], "build", timeout=900)
    if (root / "Cargo.toml").exists():
        return _run_suite(["cargo", "build"], "build", timeout=900)
    if (root / "pubspec.yaml").exists():
        return _run_suite(["flutter", "build", "apk", "--debug"], "build", timeout=1200)
    if (root / "pyproject.toml").exists():
        return _run_suite(["python", "-m", "build"], "build", timeout=600)
    raise ToolError("No recognized build system found.")
```

summarize_test_results()

```
def summarize_test_results(raw_output: str) -> dict:
    """Summarize passed, failed, skipped, and blocked checks."""
    text = raw_output
    def count(pattern: str) -> int:
        m = re.search(pattern, text)
        return int(m.group(1)) if m else 0
    summary = {
        "passed": count(r"(\d+) passed"),
        "failed": count(r"(\d+) failed"),
        "skipped": count(r"(\d+) skipped"),
        "errors": count(r"(\d+) error"),
    }
    summary["status"] = "PASS" if summary["failed"] == 0 and summary["errors"] == 0 else "FAIL"
    audit("summarize_test_results", {}, summary["status"])
    return summary
```

2.6 Planning and orchestration

Tool	Purpose & constraints
`create_plan`	Break a task into ordered steps.
`update_plan`	Mark steps complete, blocked, or changed.
`task_status`	Report current progress and blockers.
`delegate_subtask`	Assign an isolated task to a helper agent.
`collect_subtask_results`	Combine helper-agent findings.
`request_user_input`	Ask about a blocking ambiguity or destructive action.
`checkpoint`	Save a recoverable point before a risky operation.
`rollback_changes`	Restore agent-owned changes from a checkpoint.
`estimate_task_scope`	Identify likely files, commands, and checks.

create_plan()

```
def create_plan(goal: str, steps: list[str]) -> dict:
    """Break a task into ordered steps, each with a verifiable outcome."""
    plans = _load()
    plan_id = f"plan-{int(time.time())}"
    plans[plan_id] = {"goal": goal, "created": time.time(),
                      "steps": [{"step": s, "status": "pending"} for s in steps]}
    _save(plans)
    audit("create_plan", {"goal": goal[:200]}, plan_id)
    return {"plan_id": plan_id, **plans[plan_id]}
```

update_plan()

```
def update_plan(plan_id: str, step_index: int, status: str) -> dict:
    """Mark a step complete, blocked, or changed."""
    plans = _load()
    if plan_id not in plans:
        raise ToolError(f"Unknown plan: {plan_id}")
    if status not in {"pending", "in_progress", "done", "blocked", "changed"}:
        raise ToolError(f"Invalid status: {status}")
    plans[plan_id]["steps"][step_index]["status"] = status
    _save(plans)
    audit("update_plan", {"plan": plan_id, "step": step_index}, status)
    return plans[plan_id]
```

task_status()

```
def task_status(plan_id: str | None = None) -> dict:
    """Report current progress and blockers."""
    plans = _load()
    if plan_id:
        plans = {plan_id: plans.get(plan_id, {})}
    report = {}
    for pid, plan in plans.items():
        steps = plan.get("steps", [])
        report[pid] = {
            "goal": plan.get("goal"),
            "done": sum(1 for s in steps if s["status"] == "done"),
            "total": len(steps),
            "blocked": [s["step"] for s in steps if s["status"] == "blocked"],
        }
    return report
```

delegate_subtask()

```
def delegate_subtask(task: str, context: str = "") -> dict:
    """Assign an isolated task to a helper model invocation (no tool access)."""
    prompt = (f"You are a sub-agent. Complete ONLY this subtask and return your findings; "
              f"you have no file or shell access.\nTASK: {task}\nCONTEXT:\n{context[:3000]}")
    result = model_call(prompt, system="Be concise. State assumptions explicitly.")
    sub_id = f"sub-{int(time.time())}"
    state = read_json(_STATE, default={}) or {}
    state[sub_id] = {"task": task, "result": result, "collected": False}
    write_json(_STATE, state)
    audit("delegate_subtask", {"task": task[:200]}, sub_id)
    return {"subtask_id": sub_id, "result": result}
```

collect_subtask_results()

```
def collect_subtask_results(subtask_ids: list[str]) -> dict:
    """Combine helper-agent findings into one report."""
    state = read_json(_STATE, default={}) or {}
    findings = {}
    for sid in subtask_ids:
        entry = state.get(sid)
        if not entry:
            raise ToolError(f"Unknown subtask: {sid}")
        findings[sid] = entry["result"]
        entry["collected"] = True
    write_json(_STATE, state)
    audit("collect_subtask_results", {"count": len(subtask_ids)}, "collected")
    return findings
```

request_user_input()

```
def request_user_input(question: str, options: list[str] | None = None) -> dict:
    """Ask about a blocking ambiguity or destructive action; blocks until answered."""
    audit("request_user_input", {"question": question[:200]}, "awaiting_user")
    return {"question": question, "options": options or [], "status": "awaiting_user_input"}
```

checkpoint()

```
def checkpoint(label: str = "auto") -> dict:
    """Save a recoverable point (untracked-copy snapshot) before a risky operation."""
    from .core import run
    import shutil
    cid = f"cp-{int(time.time())}"
    snap = in_workspace(".agent") / "checkpoints" / cid
    snap.mkdir(parents=True, exist_ok=True)
    res = run(["git", "status", "--porcelain"])
    if res["exit_code"] == 0 and res["stdout"].strip():
        run(["git", "diff", "--binary", "HEAD"], cwd=None)
        shutil.copytree(in_workspace("."), snap / "tree",
                        ignore=shutil.ignore_patterns(".git", "node_modules", "__pycache__",
                                                      ".venv", "dist", "build", ".agent"))
    audit("checkpoint", {"label": label}, cid)
    return {"checkpoint_id": cid, "path": str(snap)}
```

rollback_changes()

```
def rollback_changes(checkpoint_id: str) -> dict:
    """Restore agent-owned changes from a checkpoint."""
    snap = in_workspace(".agent") / "checkpoints" / checkpoint_id / "tree"
    if not snap.exists():
        raise ToolError(f"Unknown checkpoint: {checkpoint_id}")
    import shutil
    for item in snap.rglob("*"):
        rel = item.relative_to(snap)
        dest = in_workspace(".") / rel
        if item.is_dir():
            dest.mkdir(parents=True, exist_ok=True)
        else:
            dest.parent.mkdir(parents=True, exist_ok=True)
            shutil.copy2(item, dest)
    audit("rollback_changes", {"checkpoint": checkpoint_id}, "restored")
    return {"restored_from": checkpoint_id}
```

estimate_task_scope()

```
def estimate_task_scope(task: str) -> dict:
    """Identify likely files, commands, and checks before editing."""
    prompt = (f"Given this task, list the files likely touched, commands to run, and checks "
              f"to verify. Return JSON: {{\"files\": [], \"commands\": [], \"checks\": []}}\n"
              f"TASK: {task}")
    import json
    import re
    raw = model_call(prompt, system="Respond with JSON only.")
    m = re.search(r"\{.*\}", raw, re.S)
    scope = json.loads(m.group(0)) if m else {"files": [], "commands": [], "checks": [], "raw": raw}
    audit("estimate_task_scope", {"task": task[:200]}, f"{len(scope.get('files', []))} files")
    return scope
```

2.7 Git and version control

Tool	Purpose & constraints
`git_status`	Show changed, staged, untracked files — run before and after edits.
`git_diff`	Inspect working-tree changes — summarize high-risk changes.
`git_log`	Review recent history — read-only by default.
`git_blame`	Inspect line history — context, not proof of intent.
`git_branch_list`	List local branches — no switching without permission.
`git_create_branch`	Create an isolated work branch with a clear name.
`git_stage`	Stage selected files — never stage unrelated user changes.
`git_commit`	Commit changes — ask first unless authorized.
`git_restore_agent_changes`	Revert only agent-made changes; preserve user edits.
`git_diff_summary`	Explain the final patch: files and behavioral effects.

git_status()

```
def git_status() -> dict:
    """Show changed, staged, and untracked files; run before and after edits."""
    res = _git("status", "--porcelain")
    files = {"modified": [], "staged": [], "untracked": []}
    for line in res["stdout"].splitlines():
        code, path = line[:2], line[3:]
        if code.strip() == "??":
            files["untracked"].append(path)
        elif code[0] != " ":
            files["staged"].append(path)
        else:
            files["modified"].append(path)
    audit("git_status", {}, f"{sum(len(v) for v in files.values())} entries")
    return files
```

git_diff()

```
def git_diff(path: str | None = None, staged: bool = False) -> str:
    """Inspect working-tree changes; high-risk changes are summarized for review."""
    args = ["diff", "--staged" if staged else "HEAD"] + ([path] if path else [])
    res = _git(*args)
    audit("git_diff", {"path": path, "staged": staged}, f"{len(res['stdout'])} chars")
    return res["stdout"][:20000]
```

git_log()

```
def git_log(limit: int = 10, path: str | None = None) -> list[str]:
    """Review recent history; read-only by default."""
    args = ["log", f"-{limit}", "--oneline"] + ([ "--", path] if path else [])
    res = _git(*args)
    return res["stdout"].splitlines()
```

git_blame()

```
def git_blame(path: str, line: int | None = None) -> str:
    """Inspect line history; use as context, not proof of intent."""
    args = ["blame", "-L", f"{line},{line}" if line else "1,200", path]
    res = _git(*args)
    return res["stdout"][:12000]
```

git_branch_list()

```
def git_branch_list() -> list[str]:
    """List local branches; never switch without permission if it risks user work."""
    res = _git("branch", "--format=%(refname:short)")
    return res["stdout"].splitlines()
```

git_create_branch()

```
def git_create_branch(name: str, base: str = "HEAD") -> dict:
    """Create an isolated work branch with a clear task name."""
    res = _git("checkout", "-b", name, base)
    if res["exit_code"] != 0:
        raise ToolError(f"Could not create branch: {res['stderr']}")
    audit("git_create_branch", {"name": name, "base": base}, "created")
    return {"branch": name, "base": base}
```

git_stage()

```
def git_stage(paths: list[str]) -> dict:
    """Stage selected files; never stage unrelated user changes."""
    res = _git("add", "--", *paths)
    audit("git_stage", {"paths": paths}, f"exit {res['exit_code']}")
    return {"staged": paths, "exit_code": res["exit_code"]}
```

git_commit()

```
def git_commit(message: str, allow_empty: bool = False, authorized: bool = False) -> dict:
    """Commit changes; ask first unless the user has explicitly authorized commits."""
    if not authorized:
        audit("git_commit", {"message": message[:200]}, "denied: not authorized")
        raise ToolError("Committing requires explicit user authorization (authorized=True).")
    args = ["commit", "-m", message] + ([] if allow_empty else ["--allow-empty=false"])
    res = _git(*args)
    audit("git_commit", {"message": message[:200]}, f"exit {res['exit_code']}")
    return {"exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])[:4000],
            "summary": git_diff_summary()}
```

git_restore_agent_changes()

```
def git_restore_agent_changes(paths: list[str] | None = None) -> dict:
    """Revert only changes made by the agent; preserve pre-existing user edits.

    Requires the agent to have recorded a checkpoint; restores from it rather
    than running a blanket `git checkout`.
    """
    from .planning import rollback_changes
    state_path = in_workspace_state()
    last = state_path.read_text(encoding="utf-8").strip() if state_path.exists() else None
    if not last:
        raise ToolError("No agent checkpoint recorded; refusing broad restore.")
    return rollback_changes(last)
```

git_diff_summary()

```
def git_diff_summary() -> dict:
    """Explain the final patch: changed files and behavioral effects."""
    diff = git_diff()
    files = _git("diff", "--name-only", "HEAD")["stdout"].splitlines()
    if not diff.strip():
        return {"files_changed": files, "summary": "No changes."}
    prompt = (f"Summarize this patch in 3-5 bullets: files changed and behavioral effects.\n"
              f"{diff[:8000]}")
    summary = model_call(prompt, system="Be precise about behavior changes.")
    audit("git_diff_summary", {"files": len(files)}, "summarized")
    return {"files_changed": files, "summary": summary}
```

2.8 Web research and documentation

Tool	Purpose & constraints
`web_search`	Search for current technical information.
`open_url`	Open a documentation page or issue.
`extract_page_text`	Extract relevant page text for analysis.
`search_official_docs`	Prioritize official documentation.
`search_repository_issues`	Find known bugs and workarounds.
`search_release_notes`	Check changes between library versions.
`fetch_api_reference`	Retrieve API details for a dependency.
`compare_sources`	Compare conflicting instructions across sources.
`save_research_note`	Store a short, cited finding in project notes.

web_search()

```
def web_search(query: str, max_results: int = 8) -> list[dict]:
    """Search for current technical information via DuckDuckGo HTML endpoint."""
    url = "https://html.duckduckgo.com/html/?q=" + urllib.parse.quote(query)
    html = _fetch(url)
    results = [{"title": _html_to_text(m.group(1)), "url": urllib.parse.unquote(m.group(2))}
               for m in re.finditer(
                   r'<a[^>]+class="result__a"[^>]+href="([^"]+)"[^>]*>(.*?)</a>', html, re.S)]
    # fallback pattern: plain anchors to duckduckgo redirect links
    if not results:
        for m in re.finditer(r'href="(https?://[^"]+)"[^>]*class="result__a"', html):
            results.append({"url": m.group(1), "title": m.group(1)})
        results = results[:max_results]
    else:
        results = [{"title": _html_to_text(t), "url": u} for u, t in
                   [(m.group(2), m.group(1)) for m in
                    re.finditer(r'<a[^>]+class="result__a"[^>]+href="([^"]+)"[^>]*>(.*?)</a>', html, re.S)]]
        results = results[:max_results]
    audit("web_search", {"query": query[:200]}, f"{len(results)} results")
    return results
```

open_url()

```
def open_url(url: str) -> str:
    """Open a documentation page or issue and return readable text."""
    if not url.startswith(("http://", "https://")):
        raise ToolError("Only http(s) URLs are allowed.")
    text = _html_to_text(_fetch(url))
    audit("open_url", {"url": url}, f"{len(text)} chars")
    return text
```

extract_page_text()

```
def extract_page_text(url: str, focus: str = "") -> str:
    """Extract relevant page text for analysis, optionally focused on a topic."""
    text = open_url(url)
    if focus:
        prompt = (f"Extract only the parts of this page relevant to: {focus}\n\n{text[:15000]}")
        text = model_call(prompt, system="Quote faithfully; omit unrelated boilerplate.")
    return text
```

search_official_docs()

```
def search_official_docs(query: str, project: str) -> str:
    """Prioritize official documentation for a query."""
    results = web_search(f"{query} site:{project}", max_results=5)
    if not results:
        results = web_search(f"{project} official documentation {query}", max_results=5)
    if not results:
        raise ToolError("No official documentation results found.")
    return extract_page_text(results[0]["url"], focus=query)
```

search_repository_issues()

```
def search_repository_issues(repo: str, query: str) -> list[dict]:
    """Find known bugs and workarounds in a repository's issue tracker (GitHub API)."""
    url = (f"https://api.github.com/search/issues?q={urllib.parse.quote(query)}"
           f"+repo:{repo}&per_page=10")
    import json
    req = urllib.request.Request(url, headers={**_UA, "Accept": "application/vnd.github+json"})
    try:
        with urllib.request.urlopen(req, timeout=20) as resp:
            data = json.loads(resp.read().decode())
    except Exception as exc:  # noqa: BLE001
        raise ToolError(f"GitHub issue search failed: {exc}") from exc
    items = [{"title": i["title"], "url": i["html_url"], "state": i["state"]}
             for i in data.get("items", [])]
    audit("search_repository_issues", {"repo": repo}, f"{len(items)} issues")
    return items
```

search_release_notes()

```
def search_release_notes(package: str, from_version: str, to_version: str) -> str:
    """Check changes between library versions."""
    url = f"https://pypi.org/pypi/{package}/json"
    import json
    try:
        with urllib.request.urlopen(url, timeout=20) as resp:
            data = json.loads(resp.read().decode())
        releases = data.get("releases", {})
        wanted = [v for v in releases if from_version <= v <= to_version][-8:]
        changelog = data["info"].get("project_urls", {}).get("Changelog")
        out = f"Versions {wanted} in range.\nChangelog URL: {changelog}"
        if changelog:
            out += "\n\n" + extract_page_text(changelog, focus=f"{from_version} to {to_version}")
        return out
    except Exception as exc:  # noqa: BLE001
        raise ToolError(f"Release note lookup failed: {exc}") from exc
```

fetch_api_reference()

```
def fetch_api_reference(url: str, symbol: str = "") -> str:
    """Retrieve API details for a dependency, focused on a symbol when given."""
    return extract_page_text(url, focus=symbol or "API reference")
```

compare_sources()

```
def compare_sources(a_url: str, b_url: str, question: str) -> str:
    """Compare conflicting instructions across sources and cite both."""
    a, b = open_url(a_url)[:12000], open_url(b_url)[:12000]
    prompt = (f"Question: {question}\n\nSOURCE A ({a_url}):\n{a}\n\nSOURCE B ({b_url}):\n{b}\n\n"
              f"Compare them, note conflicts, and state which is authoritative and why.")
    verdict = model_call(prompt, system="Be neutral and cite the sources.")
    audit("compare_sources", {"a": a_url, "b": b_url}, "compared")
    return verdict
```

save_research_note()

```
def save_research_note(note: str, title: str = "research") -> str:
    """Store a short, cited finding in project notes (markdown)."""
    p = in_workspace(".agent") / "research" / f"{re.sub(r'[^\\w-]+', '-', title)[:60]}.md"
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(f"# {title}\n\n{note}\n", encoding="utf-8")
    audit("save_research_note", {"title": title}, str(p))
    return str(p)
```

2.9 Languages, frameworks, and packages

Tool	Purpose & constraints
`detect_project_stack`	Identify languages, frameworks, package managers.
`inspect_manifest`	Read pyproject/package.json/Cargo.toml/go.mod/pubspec.yaml.
`list_dependencies`	List direct dependencies where supported.
`check_dependency_versions`	Inspect installed and required versions.
`install_dependency`	Add a dependency after permission/compat checks.
`remove_dependency`	Remove a dependency; update manifests/lockfiles.
`update_dependency`	Update a package within stated constraints.
`resolve_version_conflict`	Investigate dependency incompatibilities.
`generate_lockfile`	Produce or update a lockfile with the chosen manager.
`check_runtime_version`	Verify Python/Node/Java/Dart/Rust/Go runtimes.

detect_project_stack()

```
def detect_project_stack(root: str = ".") -> dict:
    """Identify languages, frameworks, and package managers present."""
    base = in_workspace(root)
    found = {"manifests": [], "package_manager": None, "languages": [], "frameworks": []}
    for manifest in MANIFEST_COMMANDS:
        if (base / manifest).exists():
            found["manifests"].append(manifest)
    if found["manifests"]:
        found["package_manager"] = found["manifests"][0]
    for marker, lang in (("pyproject.toml", "python"), ("package.json", "javascript"),
                         ("Cargo.toml", "rust"), ("go.mod", "go"),
                         ("pubspec.yaml", "dart"), ("pom.xml", "java")):
        if (base / marker).exists():
            found["languages"].append(lang)
    for fw, marker in (("react", "react"), ("next", "next.config"),
                       ("vite", "vite.config"), ("flutter", "flutter"),
                       ("django", "manage.py"), ("fastapi", None)):
        if marker and any(base.glob(f"{marker}*")):
            found["frameworks"].append(fw)
    audit("detect_project_stack", {"root": root}, str(found))
    return found
```

inspect_manifest()

```
def inspect_manifest(root: str = ".") -> dict:
    """Read pyproject.toml, requirements.txt, package.json, pubspec.yaml, Cargo.toml, go.mod."""
    base = in_workspace(root)
    manifests = {}
    for name in MANIFEST_COMMANDS:
        p = base / name
        if not p.exists():
            continue
        text = p.read_text(encoding="utf-8", errors="replace")
        if name.endswith(".json"):
            try:
                data = json.loads(text)
                manifests[name] = {"name": data.get("name"),
                                   "dependencies": data.get("dependencies", {}),
                                   "scripts": data.get("scripts", {})}
            except ValueError as exc:
                manifests[name] = {"error": str(exc)}
        else:
            manifests[name] = {"preview": text[:2000]}
    audit("inspect_manifest", {"root": root}, f"{len(manifests)} manifests")
    return manifests
```

list_dependencies()

```
def list_dependencies(root: str = ".") -> dict:
    """List direct dependencies where supported by the project's manifest."""
    stack = detect_project_stack(root)
    pm = stack["package_manager"]
    if not pm:
        raise ToolError("No package manifest found.")
    if pm.endswith(".json"):
        data = json.loads((in_workspace(root) / pm).read_text(encoding="utf-8"))
        return {"direct": data.get("dependencies", {}), "dev": data.get("devDependencies", {})}
    if pm == "pyproject.toml":
        text = (in_workspace(root) / pm).read_text(encoding="utf-8")
        deps = [l.strip().strip('"\'') for l in text.splitlines()
                if l.strip() and not l.startswith(("[", "#", " ")) and "==" not in l and l[:1].isalpha()]
        return {"direct": deps[:100]}
    res = run(MANIFEST_COMMANDS[pm]["lock"], timeout=120)
    return {"direct": res["stdout"].splitlines()[:200]}
```

check_dependency_versions()

```
def check_dependency_versions(root: str = ".") -> dict:
    """Inspect installed and required versions for the main runtime."""
    stack = detect_project_stack(root)
    out = {}
    if "python" in stack["languages"]:
        out["python"] = run(["python", "--version"], timeout=15)["stdout"].strip()
        res = run(["python", "-m", "pip", "list", "--format", "json"], timeout=60)
        try:
            out["installed"] = {p["name"]: p["version"] for p in json.loads(res["stdout"] or "[]")}
        except ValueError:
            out["installed"] = {}
    if "javascript" in stack["languages"]:
        out["node"] = run(["node", "--version"], timeout=15)["stdout"].strip()
        out["npm_list"] = run(["npm", "ls", "--depth", "0"], timeout=60)["stdout"][:4000]
    audit("check_dependency_versions", {}, f"{len(out)} runtimes")
    return out
```

install_dependency()

```
def install_dependency(name: str, root: str = ".", dev: bool = False,
                       approved: bool = False) -> dict:
    """Add a dependency only after permission and compatibility checks."""
    if not approved:
        audit("install_dependency", {"name": name}, "denied: needs approval")
        raise ToolError(f"Installing '{name}' requires explicit approval.")
    stack = detect_project_stack(root)
    pm = stack["package_manager"]
    if not pm:
        raise ToolError("No package manifest found.")
    cmd = list(MANIFEST_COMMANDS[pm]["install"])
    if pm == "package.json" and dev:
        cmd += ["--save-dev"]
    res = run(cmd + [name], cwd=in_workspace(root), timeout=600)
    audit("install_dependency", {"name": name}, f"exit {res['exit_code']}")
    return {"exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])[:4000]}
```

remove_dependency()

```
def remove_dependency(name: str, root: str = ".", approved: bool = False) -> dict:
    """Remove a dependency and update manifests/lockfiles carefully."""
    if not approved:
        raise ToolError(f"Removing '{name}' requires explicit approval.")
    stack = detect_project_stack(root)
    pm = stack["package_manager"]
    if not pm:
        raise ToolError("No package manifest found.")
    res = run(MANIFEST_COMMANDS[pm]["remove"] + [name], cwd=in_workspace(root), timeout=300)
    audit("remove_dependency", {"name": name}, f"exit {res['exit_code']}")
    return {"exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])[:4000]}
```

update_dependency()

```
def update_dependency(name: str, version: str | None = None, root: str = ".",
                      approved: bool = False) -> dict:
    """Update a selected package within stated constraints."""
    if not approved:
        raise ToolError(f"Updating '{name}' requires explicit approval.")
    stack = detect_project_stack(root)
    pm = stack["package_manager"]
    spec = name if not version else f"{name}=={version}" if pm.startswith("pyproject") or pm ==
"requirements.txt" else f"{name}@{version}"
    res = run(MANIFEST_COMMANDS[pm]["update"] + [spec], cwd=in_workspace(root), timeout=600)
    audit("update_dependency", {"name": name, "version": version}, f"exit {res['exit_code']}")
    return {"exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])[:4000]}
```

resolve_version_conflict()

```
def resolve_version_conflict(error_text: str) -> str:
    """Investigate dependency incompatibilities and propose minimal fixes."""
    from .debugging import diagnose_dependency_conflict
    return diagnose_dependency_conflict(error_text)["suggestion"]
```

generate_lockfile()

```
def generate_lockfile(root: str = ".") -> dict:
    """Produce or update a lockfile with the chosen package manager."""
    stack = detect_project_stack(root)
    pm = stack["package_manager"]
    if not pm:
        raise ToolError("No package manifest found.")
    res = run(MANIFEST_COMMANDS[pm]["lock"], cwd=in_workspace(root), timeout=600)
    audit("generate_lockfile", {"pm": pm}, f"exit {res['exit_code']}")
    return {"exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])[:4000]}
```

check_runtime_version()

```
def check_runtime_version(runtime: str) -> dict:
    """Verify Python, Node, Java, Dart, Rust, Go, or other runtimes are installed."""
    res = run([runtime, "--version"], timeout=15)
    out = (res["stdout"] or res["stderr"]).strip().splitlines()
    return {"runtime": runtime, "installed": res["exit_code"] == 0,
            "version": out[0] if out else None}
```

2.10 Database and backend

Tool	Purpose & constraints
`inspect_database_schema`	Inspect tables, columns, indexes, relations.
`generate_migration`	Draft a schema migration.
`validate_migration`	Check a migration for data loss/incompatibility.
`run_migration`	Apply a migration with explicit permission checks.
`query_database_readonly`	Execute a read-only query where possible.
`generate_api_endpoint`	Create a backend route/controller and logic.
`generate_api_client`	Create client functions from an API contract.
`validate_openapi_spec`	Validate an OpenAPI definition.
`generate_seed_data`	Create safe development/test seed data.
`check_service_health`	Check a local service endpoint or health route.

inspect_database_schema()

```
def inspect_database_schema(url: str) -> dict:
    """Inspect tables, columns, indexes, and relations."""
    with _connect(url) as conn:
        tables = [r[0] for r in conn.execute(
            "SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%'")]
        schema = {}
        for t in tables:
            cols = [{"name": r[1], "type": r[2], "pk": bool(r[5])}
                    for r in conn.execute(f"PRAGMA table_info('{t}')")]
            idx = [r[1] for r in conn.execute(f"PRAGMA index_list('{t}')")]
            fks = [{"column": r[3], "references": f"{r[2]}.{r[4]}"}
                   for r in conn.execute(f"PRAGMA foreign_key_list('{t}')")]
            schema[t] = {"columns": cols, "indexes": idx, "foreign_keys": fks}
    audit("inspect_database_schema", {"url": url}, f"{len(schema)} tables")
    return schema
```

generate_migration()

```
def generate_migration(name: str, change: str) -> str:
    """Draft a schema migration from a description of the change."""
    prompt = (f"Write a reversible database migration named '{name}' for: {change}\n"
              f"Provide up() and down() sections. Output only SQL or framework code.")
    migration = model_call(prompt, system="Prefer additive, non-destructive migrations.")
    audit("generate_migration", {"name": name}, "generated")
    return migration
```

validate_migration()

```
def validate_migration(sql: str) -> dict:
    """Check a migration for likely data loss or incompatibility."""
    issues = []
    for m in _DESTRUCTIVE.finditer(sql):
        issues.append(f"Destructive statement near: {sql[m.start():m.start()+60].splitlines()[0]}")
    if re.search(r"(?i)DROP\s+COLUMN", sql) and not re.search(r"(?i)down", sql, re.I):
        issues.append("DROP COLUMN without documented rollback")
    audit("validate_migration", {}, f"{len(issues)} issues")
    return {"safe": not issues, "issues": issues}
```

run_migration()

```
def run_migration(path: str, url: str, approved: bool = False) -> dict:
    """Apply a migration only with explicit environment/permission checks."""
    if not approved:
        raise ToolError("Applying a migration requires explicit approval.")
    if url.endswith("prod.db") or "production" in url:
        raise ToolError("Refusing to migrate a production database automatically.")
    sql = in_workspace(path).read_text(encoding="utf-8")
    check = validate_migration(sql)
    if not check["safe"]:
        raise ToolError(f"Migration rejected: {check['issues']}")
    conn = sqlite3.connect(url.replace("sqlite:///", ""))
    try:
        conn.executescript(sql)
        conn.commit()
    finally:
        conn.close()
    audit("run_migration", {"path": path}, "applied")
    return {"applied": True, "path": path}
```

query_database_readonly()

```
def query_database_readonly(url: str, sql: str, limit: int = 200) -> list[dict]:
    """Execute a read-only query; reject anything that mutates data."""
    if not re.match(r"(?is)^\s*(SELECT|WITH|PRAGMA|EXPLAIN)\b", sql):
        raise ToolError("Only read-only statements (SELECT/WITH/PRAGMA/EXPLAIN) are allowed.")
    if _DESTRUCTIVE.search(sql):
        raise ToolError("Statement looks destructive; rejected.")
    with _connect(url) as conn:
        conn.row_factory = sqlite3.Row
        rows = [dict(r) for r in conn.execute(sql).fetchmany(limit)]
    audit("query_database_readonly", {"sql": sql[:200]}, f"{len(rows)} rows")
    return rows
```

generate_api_endpoint()

```
def generate_api_endpoint(spec: str) -> str:
    """Create a backend route/controller and related logic from a spec."""
    prompt = (f"Implement this API endpoint (route, validation, handler, error handling) "
              f"following the project's framework conventions:\n{spec}")
    return model_call(prompt, system="Output only the code.")
```

generate_api_client()

```
def generate_api_client(spec_url: str, language: str = "python") -> str:
    """Create typed client functions from an OpenAPI contract."""
    text = extract_openapi(spec_url)
    prompt = (f"Generate a typed {language} client for this OpenAPI spec (one function per "
              f"operation, typed models, error handling):\n{text[:12000]}")
    return model_call(prompt, system="Output only the code.")
```

validate_openapi_spec()

```
def validate_openapi_spec(spec: str) -> dict:
    """Validate an OpenAPI definition's structure (openapi 3.x essentials)."""
    try:
        data = json.loads(extract_openapi(spec))
    except ValueError as exc:
        return {"valid": False, "errors": [f"Invalid JSON: {exc}"]}
    errors = []
    if not str(data.get("openapi", "")).startswith("3"):
        errors.append("Missing or unsupported 'openapi' version (expected 3.x)")
    if not data.get("info"):
        errors.append("Missing 'info' object")
    if not data.get("paths"):
        errors.append("Missing 'paths' object")
    audit("validate_openapi_spec", {}, f"{len(errors)} errors")
    return {"valid": not errors, "errors": errors}
```

generate_seed_data()

```
def generate_seed_data(model: str, count: int = 20) -> str:
    """Create safe development/test seed data (never real user data)."""
    prompt = (f"Generate {count} rows of fake seed data for '{model}' as SQL INSERTs. "
              f"Use obviously fake values (example.com, 555 numbers). Output only SQL.")
    return model_call(prompt, system="Data must be synthetic and safe for tests.")
```

check_service_health()

```
def check_service_health(url: str, timeout: int = 10) -> dict:
    """Check a local service endpoint or health route."""
    try:
        with urllib.request.urlopen(url, timeout=timeout) as resp:
            status = resp.status
            body = resp.read(2000).decode("utf-8", errors="replace")
    except Exception as exc:  # noqa: BLE001
        audit("check_service_health", {"url": url}, "unhealthy")
        return {"url": url, "healthy": False, "error": str(exc)}
    audit("check_service_health", {"url": url}, f"HTTP {status}")
    return {"url": url, "healthy": status < 400, "status": status, "body": body[:500]}
```

2.11 LLM and model management

Tool	Purpose & constraints
`list_models`	List configured models and their capabilities.
`select_model`	Choose a model by task, cost, speed, context.
`call_model`	Send a structured prompt to a configured model.
`call_model_with_tools`	Allow a model to request approved tool calls.
`estimate_tokens`	Estimate context/token usage.
`summarize_context`	Condense older conversation or tool output.
`route_task`	Route coding/planning/review to suitable models.
`validate_model_output`	Check output shape and required fields.
`record_model_usage`	Track latency, token use, errors (no secrets).
`fallback_model`	Switch to an approved backup model after failure.

list_models()

```
def list_models() -> dict:
    """List configured models and their capabilities."""
    configured = {"base_url": os.environ.get("LLM_BASE_URL"),
                  "model": os.environ.get("LLM_MODEL", "gpt-4o-mini"),
                  "has_key": bool(os.environ.get("LLM_API_KEY"))}
    audit("list_models", {}, "listed")
    return {"active": configured, "catalog": _CATALOG}
```

select_model()

```
def select_model(task: str, cost_preference: str = "balanced") -> str:
    """Choose a model based on task type, cost, speed, and context size."""
    best = _ROUTING.get(task, "gpt-4o-mini")
    if cost_preference == "cheapest":
        best = "gpt-4o-mini"
    elif cost_preference == "best" and task in ("plan", "review"):
        best = "gpt-4o"
    audit("select_model", {"task": task, "pref": cost_preference}, best)
    return best
```

call_model()

```
def call_model(prompt: str, system: str | None = None, model: str | None = None) -> str:
    """Send a structured prompt to the configured model and record usage."""
    started = time.time()
    text = model_call(prompt, system=system, model=model)
    _record({"ts": started, "model": model or os.environ.get("LLM_MODEL"),
             "latency_ms": int((time.time() - started) * 1000),
             "prompt_chars": len(prompt), "response_chars": len(text), "error": None})
    return text
```

call_model_with_tools()

```
def call_model_with_tools(prompt: str, tools: list[dict], registry: dict,
                          max_rounds: int = 5) -> dict:
    """Allow a model to request approved tool calls; validates every argument."""
    from .security import verify_command_allowlist  # noqa: F401  (policy import)
    transcript = [{"role": "user", "content": prompt}]
    calls = []
    for _ in range(max_rounds):
        raw = call_model(
            prompt=json.dumps({"messages": transcript, "tools": tools}),
            system="Request tool calls by responding with JSON: "
                   '{"tool": name, "args": {...}} or {"final": "answer"}.')
        try:
            decision = json.loads(raw)
        except ValueError:
            return {"final": raw, "tool_calls": calls}
        if "final" in decision:
            return {"final": decision["final"], "tool_calls": calls}
        name, args = decision.get("tool"), decision.get("args", {})
        fn = registry.get(name)
        if fn is None:
            transcript.append({"role": "system", "content": f"Unknown tool '{name}'."})
            continue
        try:
            result = fn(**args)
        except Exception as exc:  # noqa: BLE001 - tool failures feed back to the model
            result = {"error": str(exc)}
        calls.append({"tool": name, "args": args, "result": str(result)[:2000]})
        transcript.append({"role": "tool", "content": str(result)[:4000]})
    return {"final": "Max tool rounds reached.", "tool_calls": calls}
```

estimate_tokens()

```
def estimate_tokens(text: str) -> int:
    """Estimate token usage (~4 chars/token heuristic; use an exact tokenizer if available)."""
    try:
        import tiktoken
        enc = tiktoken.get_encoding("cl100k_base")
        return len(enc.encode(text))
    except ImportError:
        return len(text) // 4
```

summarize_context()

```
def summarize_context(items: list[str], keep_ratio: float = 0.3) -> dict:
    """Condense older conversation or tool output, keeping the most recent items."""
    keep_from = int(len(items) * (1 - keep_ratio))
    kept = items[keep_from:]
    dropped = items[:keep_from]
    summary = ""
    if dropped:
        summary = call_model("Summarize these earlier messages in 5 bullets, keeping "
                             "decisions, file paths, and unresolved issues:\n" +
                             "\n".join(dropped)[-12000],
                             system="Be concise and factual.")
    audit("summarize_context", {"dropped": len(dropped), "kept": len(kept)}, "summarized")
    return {"summary": summary, "kept": kept}
```

route_task()

```
def route_task(task_description: str) -> dict:
    """Route coding, planning, and review tasks to suitable models."""
    import re
    text = task_description.lower()
    kind = ("plan" if any(w in text for w in ("plan", "design", "architect"))
            else "review" if any(w in text for w in ("review", "audit", "critique"))
            else "research" if any(w in text for w in ("research", "find", "look up"))
            else "code")
    model = select_model(kind)
    audit("route_task", {"kind": kind}, model)
    return {"task_kind": kind, "model": model}
```

validate_model_output()

```
def validate_model_output(text: str, required_fields: list[str]) -> dict:
    """Check output shape and required fields before trusting structured output."""
    import re
    errors = []
    try:
        data = json.loads(re.search(r"\{.*\}|\[.*\]", text, re.S).group(0))
    except (ValueError, AttributeError):
        return {"valid": False, "errors": ["Output is not valid JSON."]}
    if isinstance(data, dict):
        for f in required_fields:
            if f not in data:
                errors.append(f"Missing field: {f}")
    audit("validate_model_output", {"fields": required_fields}, f"{len(errors)} errors")
    return {"valid": not errors, "errors": errors, "data": data}
```

record_model_usage()

```
def record_model_usage(entry: dict) -> str:
    """Track latency, token use, and errors without storing secrets."""
    _record({"ts": time.time(), **entry})
    return f"Recorded usage ({_USAGE_LOG})"
```

fallback_model()

```
def fallback_model(primary_error: str, task: str) -> str:
    """Switch to an approved backup model after a failure."""
    for candidate in ("gpt-4o-mini", "gemini-flash"):
        if candidate != os.environ.get("LLM_MODEL"):
            audit("fallback_model", {"error": primary_error[:120]}, candidate)
            return candidate
    raise ToolError("No fallback model configured.")
```

2.12 Code intelligence

Tool	Purpose & constraints
`parse_source_file`	Parse a language into a syntax structure.
`find_symbol`	Locate a class, function, variable, or method.
`find_symbol_references`	Find usages across the repository.
`go_to_definition`	Navigate from a reference to its definition.
`get_type_information`	Retrieve type information from an LSP.
`language_server_request`	Access supported LSP features.
`build_dependency_graph`	Map modules and their dependencies.
`index_codebase`	Build a searchable project index.
`semantic_code_search`	Retrieve code by meaning, not just text.
`explain_code_region`	Explain a function or module with context.

parse_source_file()

```
def parse_source_file(path: str) -> dict:
    """Parse a supported language into a structure of top-level symbols."""
    p = in_workspace(path)
    text = p.read_text(encoding="utf-8", errors="replace")
    ext = p.suffix
    if ext == ".py" and which("python"):
        import ast
        try:
            tree = ast.parse(text)
            symbols = [{"kind": type(n).__name__, "name": n.name, "line": n.lineno}
                       for n in tree.body if hasattr(n, "name")]
            return {"language": "python", "symbols": symbols}
        except SyntaxError as exc:
            return {"language": "python", "error": str(exc)}
    pattern = {"javascript": r"^\s*(?:export\s+)?(?:async\s+)?function\s+(\w+)|^\s*(?:export\s+)?class\s+
(\w+)",
               "typescript": r"^\s*(?:export\s+)?(?:async\s+)?function\s+(\w+)|^\s*(?:export\s+)?
(?:abstract\s+)?(?:class|interface|type)\s+(\w+)",
               "go": r"^func\s+(?:\([^)]+\)\s+)?(\w+)|^type\s+(\w+)",
               "rust": r"^\s*(?:pub\s+)?(?:fn|struct|enum|trait)\s+(\w+)",
               }.get(_CODE_EXT.get(ext, ""), None)
    if not pattern:
        raise ToolError(f"Unsupported file type for parsing: {ext}")
    symbols = []
    for n, line in enumerate(text.splitlines(), 1):
        m = re.match(pattern, line)
        if m:
            symbols.append({"kind": "definition", "name": next(g for g in m.groups() if g), "line": n})
    audit("parse_source_file", {"path": path}, f"{len(symbols)} symbols")
    return {"language": _CODE_EXT.get(ext, ext), "symbols": symbols}
```

find_symbol()

```
def find_symbol(name: str, path: str = ".") -> list[dict]:
    """Locate a class, function, variable, or method definition."""
    pattern = (rf"^\s*(?:export\s+)?(?:async\s+)?
(?:def|function|class|interface|type|struct|enum|trait|fn)\s+{re.escape(name)}\b"
               rf"|^\s*(?:export\s+)?(?:const|let|var|public|private|static)\s+{re.escape(name)}\b")
    res = run(["rg", "-n", "--no-heading", "-e", pattern, path], timeout=60) if which("rg") else None
    if res and res["stdout"]:
        return [{"location": l.split(":")[0] + ":" + l.split(":")[1], "line": l.split(":", 2)[2].strip()
[:120]}
                for l in res["stdout"].splitlines()[:50]]
    from .filesystem import search_files
    hits = search_files(rf"(def|function|class|struct|enum|trait|fn)\s+{re.escape(name)}\b",
                        path, regex=True, max_results=50)
    return [{"location": h.split(":")[0] + ":" + h.split(":")[1], "line": h.split(":", 2)[2].strip()[:120]}
            for h in hits]
```

find_symbol_references()

```
def find_symbol_references(symbol: str, path: str = ".") -> list[str]:
    """Find usages of a symbol across the repository (word-boundary safe)."""
    from .filesystem import find_references
    refs = find_references(symbol, path)
    audit("find_symbol_references", {"symbol": symbol}, f"{len(refs)} refs")
    return refs
```

go_to_definition()

```
def go_to_definition(symbol: str, path: str = ".") -> dict:
    """Navigate from a reference to its definition."""
    defs = find_symbol(symbol, path)
    if not defs:
        raise ToolError(f"No definition found for '{symbol}'.")
    return {"symbol": symbol, "definition": defs[0]}
```

get_type_information()

```
def get_type_information(symbol: str, path: str = ".") -> dict:
    """Retrieve type information from a language server where configured."""
    cmd = os.environ.get("LSP_COMMAND")
    if not cmd:
        defs = find_symbol(symbol, path)
        return {"symbol": symbol, "types": "LSP not configured (set LSP_COMMAND).",
                "definition_guess": defs[:1] if defs else None}
    # A full LSP handshake is project-specific; delegate to the configured server.
    res = run(cmd.split() + [symbol, path], timeout=30)
    return {"symbol": symbol, "raw": res["stdout"][:4000] or res["stderr"][:2000]}
```

language_server_request()

```
def language_server_request(method: str, params: dict, root: str = ".") -> dict:
    """Access supported Language Server Protocol features via the configured server."""
    cmd = os.environ.get("LSP_COMMAND")
    if not cmd:
        raise ToolError("No LSP configured; set LSP_COMMAND (e.g. 'pyls').")
    payload = {"jsonrpc": "2.0", "id": 1, "method": method, "params": params}
    import subprocess
    proc = subprocess.run(cmd.split(), input=json.dumps(payload) + "\n",
                          capture_output=True, text=True, timeout=30, cwd=in_workspace(root))
    return {"stdout": proc.stdout[:6000], "stderr": proc.stderr[:2000]}
```

build_dependency_graph()

```
def build_dependency_graph(path: str = ".") -> dict:
    """Map modules and their dependencies from import statements."""
    from .filesystem import list_files
    graph: dict[str, list[str]] = {}
    import_re = {"python": re.compile(r"^\s*(?:from\s+([\w.]+)|import\s+([\w.]+))"),
                 "javascript": re.compile(r"""^\s*(?:import|from)\s+['"](\.?[\w./-]+)['"]"""),
                 "typescript": re.compile(r"""^\s*(?:import|from)\s+['"](\.?[\w./-]+)['"]""")}
    for rel in list_files(path):
        ext = os.path.splitext(rel)[1]
        lang = _CODE_EXT.get(ext)
        if not lang or lang not in import_re:
            continue
        p = in_workspace(rel if path == "." else f"{path}/{rel}")
        deps = []
        for line in p.read_text(encoding="utf-8", errors="replace").splitlines():
            m = import_re[lang].match(line)
            if m:
                deps.append(next(g for g in m.groups() if g))
        if deps:
            graph[rel] = sorted(set(deps))[:40]
    audit("build_dependency_graph", {"path": path}, f"{len(graph)} modules")
    return graph
```

index_codebase()

```
def index_codebase(path: str = ".") -> dict:
    """Build a searchable project index of files and their symbols."""
    from .filesystem import list_files
    index = {}
    for rel in list_files(path):
        p = in_workspace(rel if path == "." else f"{path}/{rel}")
        if p.suffix not in _CODE_EXT or p.stat().st_size > 200_000:
            continue
        try:
            parsed = parse_source_file(str(p))
            index[rel] = [s["name"] for s in parsed.get("symbols", [])]
        except (ToolError, OSError):
            continue
    write_json(_INDEX, index)
    audit("index_codebase", {"path": path}, f"{len(index)} files indexed")
    return {"files_indexed": len(index), "symbols": sum(len(v) for v in index.values())}
```

semantic_code_search()

```
def semantic_code_search(query: str, top_k: int = 5) -> list[dict]:
    """Retrieve relevant code by meaning where embeddings are configured, else keyword search."""
    index = read_json(_INDEX)
    if not index:
        index_codebase()
        index = read_json(_INDEX, default={})
    keywords = set(re.findall(r"\w+", query.lower()))
    scored = []
    for file, symbols in (index or {}).items():
        score = sum(1 for s in symbols if any(k in s.lower() for k in keywords))
        if score:
            scored.append({"file": file, "score": score, "symbols": symbols[:10]})
    scored.sort(key=lambda x: -x["score"])
    audit("semantic_code_search", {"query": query[:120]}, f"{len(scored)} hits")
    return scored[:top_k]
```

explain_code_region()

```
def explain_code_region(path: str, start_line: int, end_line: int) -> str:
    """Explain a selected function or module using nearby context."""
    from .filesystem import read_file_range
    snippet = read_file_range(path, start_line, end_line)
    prompt = (f"Explain what this code does, its inputs/outputs, and any risks.\n"
              f"FILE: {path}\n{snippet}")
    explanation = model_call(prompt, system="Be precise and technical.")
    audit("explain_code_region", {"path": path, "lines": [start_line, end_line]}, "explained")
    return explanation
```

2.13 Frontend and UI

Tool	Purpose & constraints
`inspect_ui_project`	Detect app framework and entry points.
`generate_component`	Create a UI component using conventions.
`edit_component`	Modify a selected component.
`run_dev_server`	Start a local frontend development server.
`capture_screenshot`	Capture a local UI screenshot.
`inspect_console_errors`	Collect browser console errors.
`inspect_network_failures`	Identify failed frontend requests.
`run_accessibility_check`	Check supported accessibility rules.
`compare_ui_screenshots`	Compare a result with a reference image.
`validate_responsive_layout`	Test supported viewport sizes.

inspect_ui_project()

```
def inspect_ui_project(path: str = ".") -> dict:
    """Detect app framework and entry points."""
    from .packages import detect_project_stack
    root = in_workspace(path)
    stack = detect_project_stack(path)
    entries = [str(p.relative_to(root)) for p in
               (root / "src").glob("main.*")] + [str(p.relative_to(root)) for p in root.glob("main.*")]
    if (root / "lib" / "main.dart").exists():
        entries.append("lib/main.dart")
    audit("inspect_ui_project", {"path": path}, f"frameworks={stack.get('frameworks')}")
    return {"frameworks": stack.get("frameworks", []), "entry_points": entries[:10]}
```

generate_component()

```
def generate_component(name: str, framework: str, conventions: str = "") -> str:
    """Create a UI component using the project's conventions."""
    prompt = (f"Create a {framework} component named {name} following these project "
              f"conventions:\n{conventions or '(infer standard best practices)'}\n"
              f"Include accessibility labels. Output only code.")
    return model_call(prompt, system="Follow modern framework idioms and a11y best practices.")
```

edit_component()

```
def edit_component(path: str, old: str, new: str) -> dict:
    """Modify a selected component with a validated edit."""
    from .filesystem import apply_patch
    return apply_patch(path, old, new)
```

run_dev_server()

```
def run_dev_server(command: list[str], name: str = "dev-server") -> dict:
    """Start a local frontend development server and report its log location."""
    audit("run_dev_server", {"command": command}, "starting")
    return _start_process(command, name=name)
```

capture_screenshot()

```
def capture_screenshot(url: str, out_path: str = ".agent/screenshot.png",
                       width: int = 1280, height: int = 800) -> str:
    """Capture a local UI screenshot where browser automation is available."""
    pw = _playwright()
    with pw() as p:
        browser = p.chromium.launch()
        page = browser.new_page(viewport={"width": width, "height": height})
        page.goto(url, wait_until="networkidle", timeout=30000)
        target = in_workspace(out_path)
        target.parent.mkdir(parents=True, exist_ok=True)
        page.screenshot(path=str(target), full_page=True)
        browser.close()
    audit("capture_screenshot", {"url": url}, str(target))
    return str(target)
```

inspect_console_errors()

```
def inspect_console_errors(url: str, wait_ms: int = 3000) -> list[str]:
    """Collect browser console errors from an authorized test session."""
    pw = _playwright()
    errors: list[str] = []
    with pw() as p:
        browser = p.chromium.launch()
        page = browser.new_page()
        page.on("console", lambda m: errors.append(f"[{m.type}] {m.text}") if m.type == "error" else None)
        page.on("pageerror", lambda e: errors.append(f"[pageerror] {e}"))
        page.goto(url, timeout=30000)
        page.wait_for_timeout(wait_ms)
        browser.close()
    audit("inspect_console_errors", {"url": url}, f"{len(errors)} errors")
    return errors
```

inspect_network_failures()

```
def inspect_network_failures(url: str, wait_ms: int = 5000) -> list[dict]:
    """Identify failed frontend requests in an authorized test session."""
    pw = _playwright()
    failures: list[dict] = []
    with pw() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        def on_response(resp):
            if resp.status >= 400:
                failures.append({"url": resp.url, "status": resp.status})
        page.on("response", on_response)
        page.goto(url, timeout=30000)
        page.wait_for_timeout(wait_ms)
        browser.close()
    audit("inspect_network_failures", {"url": url}, f"{len(failures)} failures")
    return failures
```

run_accessibility_check()

```
def run_accessibility_check(url: str) -> list[str]:
    """Check supported accessibility rules via axe-core if present."""
    pw = _playwright()
    axe = in_workspace("node_modules/axe-core/axe.min.js")
    if not axe.exists():
        raise ToolError("axe-core not found; run 'npm i -D axe-core' in the project first.")
    violations: list[str] = []
    with pw() as p:
        browser = p.chromium.launch()
        page = browser.new_page()
        page.goto(url, timeout=30000)
        page.add_script_tag(path=str(axe))
        results = page.evaluate("async () => await axe.run()")
        for v in results.get("violations", []):
            violations.append(f"[{v['impact']}] {v['id']}: {v['help']} ({len(v['nodes'])} nodes)")
        browser.close()
    audit("run_accessibility_check", {"url": url}, f"{len(violations)} violations")
    return violations
```

compare_ui_screenshots()

```
def compare_ui_screenshots(image_a: str, image_b: str) -> dict:
    """Compare a result with a reference image pixel-by-pixel."""
    try:
        from PIL import Image, ImageChops
    except ImportError as exc:
        raise ToolError("Pillow not installed (pip install pillow).") from exc
    a = Image.open(in_workspace(image_a)).convert("RGB")
    b = Image.open(in_workspace(image_b)).convert("RGB")
    if a.size != b.size:
        return {"match": False, "reason": f"Size differs: {a.size} vs {b.size}"}
    diff = ImageChops.difference(a, b)
    changed = sum(1 for px in diff.getdata() if px != (0, 0, 0))
    ratio = changed / (a.size[0] * a.size[1])
    audit("compare_ui_screenshots", {}, f"{ratio:.2%} pixels differ")
    return {"match": ratio < 0.01, "differing_pixel_ratio": round(ratio, 4)}
```

validate_responsive_layout()

```
def validate_responsive_layout(url: str,
                               viewports: list[tuple[int, int]] | None = None) -> list[dict]:
    """Test supported viewport sizes for horizontal overflow and load failures."""
    viewports = viewports or [(375, 812), (768, 1024), (1440, 900)]
    pw = _playwright()
    results = []
    with pw() as p:
        browser = p.chromium.launch()
        for width, height in viewports:
            page = browser.new_page(viewport={"width": width, "height": height})
            page.goto(url, timeout=30000)
            overflow = page.evaluate(
                "() => document.documentElement.scrollWidth > document.documentElement.clientWidth")
            results.append({"viewport": f"{width}x{height}", "horizontal_overflow": overflow})
            page.close()
        browser.close()
    audit("validate_responsive_layout", {"url": url}, f"{sum(r['horizontal_overflow'] for r in results)}
issues")
    return results
```

2.14 Build, run, and deployment

Tool	Purpose & constraints
`build_project`	Run the appropriate project build.
`run_application`	Launch the app in a selected environment.
`stop_application`	Stop an agent-started app process.
`inspect_build_artifacts`	List and validate generated artifacts.
`package_application`	Produce a distributable package.
`check_environment_config`	Validate required config keys without exposing values.
`build_container`	Build a container image when configured.
`run_container_tests`	Test a containerized application.
`deploy_preview`	Deploy to an isolated preview if authorized.
`deploy_production`	Production deployment — explicit confirmation only.
`rollback_deployment`	Roll back an authorized deployment.

build_project()

```
def build_project(path: str = ".", timeout: int = 900) -> dict:
    """Run the appropriate project build and report artifacts or errors."""
    root = in_workspace(path)
    cmd = _build_command(root)
    if not cmd:
        raise ToolError("No recognized build command for this project.")
    res = run(cmd, cwd=root, timeout=timeout)
    audit("build_project", {"command": cmd}, f"exit {res['exit_code']}")
    return {"command": cmd, "exit_code": res["exit_code"],
            "output": (res["stdout"] + res["stderr"])[-12000:]}
```

run_application()

```
def run_application(command: list[str], name: str = "app") -> dict:
    """Launch the app in a selected environment as an agent-owned process."""
    audit("run_application", {"command": command}, "started")
    return start_process(command, name=name)
```

stop_application()

```
def stop_application(name: str) -> dict:
    """Stop an agent-started app process only."""
    return stop_process(name)
```

inspect_build_artifacts()

```
def inspect_build_artifacts(dir_path: str = "dist") -> list[dict]:
    """List and validate generated artifacts (size, presence, timestamps)."""
    root = in_workspace(dir_path)
    if not root.exists():
        raise ToolError(f"Artifact directory not found: {root}")
    artifacts = []
    for p in sorted(root.rglob("*")):
        if p.is_file():
            artifacts.append({"file": str(p.relative_to(root)),
                              "size": p.stat().st_size, "modified": p.stat().st_mtime})
    audit("inspect_build_artifacts", {"dir": dir_path}, f"{len(artifacts)} artifacts")
    return artifacts[:200]
```

package_application()

```
def package_application(path: str = ".", output: str = "dist") -> dict:
    """Produce a distributable package using the project's own tooling."""
    res = build_project(path)
    if res["exit_code"] != 0:
        raise ToolError("Build failed; refusing to package a broken application.")
    root = in_workspace(path)
    out = in_workspace(output)
    import shutil
    import time
    archive = shutil.make_archive(str(out / f"release-{int(time.time())}"), "zip",
                                  root_dir=root,
                                  base_dir=None if not (root / "src").exists() else "src")
    audit("package_application", {"output": output}, archive)
    return {"archive": archive, "size": os.path.getsize(archive)}
```

check_environment_config()

```
def check_environment_config(required_keys: list[str]) -> dict:
    """Validate required configuration keys exist without exposing their values."""
    missing = [k for k in required_keys if not os.environ.get(k)]
    placeholders = [k for k in required_keys
                    if os.environ.get(k, "").lower() in ("changeme", "xxx", "your-key-here", "")]
    audit("check_environment_config", {"keys": required_keys}, f"{len(missing)} missing")
    return {"missing": missing, "placeholder_values": placeholders,
            "ok": not missing and not placeholders}
```

build_container()

```
def build_container(path: str = ".", tag: str = "agent-app:latest") -> dict:
    """Build a container image when the project is containerized."""
    root = in_workspace(path)
    if not (root / "Dockerfile").exists():
        raise ToolError("No Dockerfile in the project.")
    docker = which("docker") or which("podman")
    if not docker:
        raise ToolError("Neither docker nor podman is installed.")
    res = run([docker, "build", "-t", tag, str(root)], timeout=1800)
    audit("build_container", {"tag": tag}, f"exit {res['exit_code']}")
    return {"exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])[-8000:]}
```

run_container_tests()

```
def run_container_tests(path: str = ".", tag: str = "agent-app:latest") -> dict:
    """Test a containerized application by running its test command inside the image."""
    docker = which("docker") or which("podman")
    if not docker:
        raise ToolError("Neither docker nor podman is installed.")
    root = in_workspace(path)
    res = run([docker, "run", "--rm", "--entrypoint", "sh", tag, "-c",
               "python -m pytest -q || npm test -- --watchAll=false || true"], cwd=root, timeout=900)
    audit("run_container_tests", {"tag": tag}, f"exit {res['exit_code']}")
    return {"exit_code": res["exit_code"], "output": (res["stdout"] + res["stderr"])[-8000:]}
```

deploy_preview()

```
def deploy_preview(url: str, token_env: str = "PREVIEW_DEPLOY_TOKEN") -> dict:
    """Deploy to an isolated preview environment if authorized via env token."""
    token = os.environ.get(token_env)
    if not token:
        raise ToolError(f"No deploy token in ${token_env}; preview deploy not authorized.")
    import urllib.request
    req = urllib.request.Request(url, method="POST",
                                 headers={"Authorization": f"Bearer {token}"})
    try:
        with urllib.request.urlopen(req, timeout=120) as resp:
            body = resp.read(2000).decode("utf-8", errors="replace")
        audit("deploy_preview", {"url": url}, "deployed")
        return {"deployed": True, "response": body[:1000]}
    except Exception as exc:  # noqa: BLE001
        raise ToolError(f"Preview deploy failed: {exc}") from exc
```

deploy_production()

```
def deploy_production(url: str, confirmed: bool = False, token_env: str = "PROD_DEPLOY_TOKEN") -> dict:
    """Production deployment; always requires explicit confirmation."""
    if not confirmed:
        audit("deploy_production", {"url": url}, "denied: not confirmed")
        raise ToolError("Production deployment requires confirmed=True from the user.")
    return deploy_preview(url, token_env=token_env)
```

rollback_deployment()

```
def rollback_deployment(previous_artifact: str, url: str) -> dict:
    """Roll back an authorized deployment to a prior verified artifact."""
    artifact = in_workspace(previous_artifact)
    if not artifact.exists():
        raise ToolError(f"Rollback artifact not found: {artifact}")
    audit("rollback_deployment", {"artifact": previous_artifact, "url": url}, "rolled back")
    return {"rolled_back_to": str(artifact), "target": url,
            "note": "Upload/activate the artifact with your platform's release tool."}
```

2.15 Security and permissions

Tool	Purpose & constraints
`inspect_secrets_presence`	Detect likely secrets without printing values.
`redact_sensitive_output`	Remove tokens, passwords, personal data.
`audit_dependencies`	Run an approved vulnerability checker.
`scan_code_security`	Run configured static security analysis.
`check_file_permissions`	Inspect permissions on relevant files.
`validate_path_scope`	Ensure a path stays inside approved roots.
`request_permission`	Ask for approval before a restricted action.
`audit_tool_action`	Record action, scope, outcome (no secret content).
`verify_command_allowlist`	Check a command against the configured policy.
`verify_network_policy`	Enforce allowed hosts / network rules.

inspect_secrets_presence()

```
def inspect_secrets_presence(path: str = ".") -> list[dict]:
    """Detect likely secrets WITHOUT printing their values."""
    from .filesystem import list_files
    findings = []
    for rel in list_files(path):
        p = in_workspace(rel if path == "." else f"{path}/{rel}")
        try:
            if p.stat().st_size > 500_000:
                continue
            text = p.read_text(encoding="utf-8", errors="replace")
        except OSError:
            continue
        for label, pat in _SECRET_PATTERNS:
            if pat.search(text):
                findings.append({"file": rel, "secret_type": label, "value": "[REDACTED]"})
    audit("inspect_secrets_presence", {"path": path}, f"{len(findings)} findings")
    return findings
```

redact_sensitive_output()

```
def redact_sensitive_output(text: str) -> str:
    """Remove tokens, passwords, and personal data from logs."""
    out = text
    for _, pat in _SECRET_PATTERNS:
        out = pat.sub("[REDACTED]", out)
    out = re.sub(r"(?i)(authorization:\s*bearer\s+)\S+", r"\1[REDACTED]", out)
    out = re.sub(r"(?i)([\w.+-]+@[\w-]+\.[\w.]+)", "[EMAIL-REDACTED]", out)
    return out
```

audit_dependencies()

```
def audit_dependencies(root: str = ".") -> dict:
    """Run an approved dependency vulnerability checker."""
    if which("pip-audit"):
        res = run(["python", "-m", "pip_audit", "-r", "requirements.txt"], cwd=root, timeout=300)
        return {"tool": "pip-audit", "exit_code": res["exit_code"], "report": res["stdout"][:8000]}
    if which("npm"):
        res = run(["npm", "audit", "--json"], cwd=root, timeout=300)
        return {"tool": "npm audit", "exit_code": res["exit_code"], "report": res["stdout"][:8000]}
    raise ToolError("No dependency auditor found (tried pip-audit, npm audit).")
```

scan_code_security()

```
def scan_code_security(path: str = ".") -> dict:
    """Run configured static security analysis."""
    if which("bandit"):
        res = run(["bandit", "-r", path, "-f", "json"], timeout=600)
        try:
            data = json.loads(res["stdout"] or "{}")
            issues = [{"file": r.get("filename"), "issue": r.get("issue_text"),
                       "severity": r.get("issue_severity")}
                      for r in data.get("results", [])[:50]]
        except ValueError:
            issues = [{"raw": res["stdout"][:4000]}]
        return {"tool": "bandit", "issues": issues}
    if which("gitleaks"):
        res = run(["gitleaks", "detect", "--report-format", "json", "--no-banner"], timeout=600)
        return {"tool": "gitleaks", "exit_code": res["exit_code"], "report": res["stdout"][:8000]}
    raise ToolError("No security scanner found (tried bandit, gitleaks).")
```

check_file_permissions()

```
def check_file_permissions(path: str) -> dict:
    """Inspect permissions on relevant files; flag world-writable or secret files."""
    import stat as stat_mod
    p = in_workspace(path)
    mode = p.stat().st_mode
    perms = stat_mod.filemode(mode)
    risky = bool(mode & stat_mod.S_IWOTH) or p.name.startswith(".env")
    audit("check_file_permissions", {"path": path}, f"{perms} risky={risky}")
    return {"path": str(p), "permissions": perms, "risky": risky,
            "note": "World-writable files or loose .env permissions are dangerous." if risky else None}
```

validate_path_scope()

```
def validate_path_scope(path: str) -> dict:
    """Ensure a requested path stays inside approved roots."""
    try:
        resolved = in_workspace(path)
        return {"allowed": True, "resolved": str(resolved)}
    except ToolError as exc:
        return {"allowed": False, "error": str(exc)}
```

request_permission()

```
def request_permission(action: str, detail: str = "") -> dict:
    """Ask for approval before a restricted action (blocks the agent loop)."""
    audit("request_permission", {"action": action, "detail": detail[:200]}, "awaiting_approval")
    return {"action": action, "detail": detail, "status": "awaiting_approval"}
```

audit_tool_action()

```
def audit_tool_action(tool: str, args: dict, outcome: str) -> str:
    """Record the action, scope, and outcome without secret content."""
    from .core import _redact
    audit(tool, _redact(args), outcome)
    return f"Logged {tool}: {outcome}"
```

verify_command_allowlist()

```
def verify_command_allowlist(command: list[str]) -> dict:
    """Check a command against the configured policy."""
    from .terminal import verify_command_allowlist as _check
    allowed = _check(command)
    audit("verify_command_allowlist", {"command": command[:3]}, "allowed" if allowed else "denied")
    return {"command": command[0] if command else None, "allowed": allowed}
```

verify_network_policy()

```
def verify_network_policy(url: str, allow_hosts: list[str] | None = None) -> dict:
    """Enforce allowed hosts or network access rules."""
    from urllib.parse import urlparse
    host = urlparse(url).hostname or ""
    allowed_hosts = allow_hosts or [
        h for h in os.environ.get("AGENT_ALLOWED_HOSTS", "").split(",") if h]
    if not allowed_hosts:
        audit("verify_network_policy", {"url": url}, "allowed: no policy configured")
        return {"allowed": True, "note": "No AGENT_ALLOWED_HOSTS policy configured."}
    ok = any(host == h or host.endswith("." + h) for h in allowed_hosts)
    audit("verify_network_policy", {"host": host}, "allowed" if ok else "denied")
    return {"allowed": ok, "host": host, "policy": allowed_hosts}
```

2.16 Memory and project management

Tool	Purpose & constraints
`read_project_instructions`	Read AGENTS.md or project guidance files.
`read_project_manifest`	Summarize project purpose and structure.
`load_task_context`	Load only files/notes relevant to the task.
`save_project_note`	Save stable project-specific facts.
`search_project_notes`	Retrieve relevant notes from prior work.
`update_task_state`	Track task status, decisions, blockers.
`record_decision`	Store a short architectural decision + rationale.
`clear_task_memory`	Remove temporary context after a task.
`summarize_session`	Produce a concise handoff summary.

read_project_instructions()

```
def read_project_instructions(root: str = ".") -> dict:
    """Read AGENTS.md or other project-specific guidance files."""
    base = in_workspace(root)
    found = {}
    for name in _INSTRUCTION_FILES:
        p = base / name
        if p.exists():
            found[name] = p.read_text(encoding="utf-8", errors="replace")[:8000]
    audit("read_project_instructions", {"root": root}, f"{len(found)} files")
    return found
```

read_project_manifest()

```
def read_project_manifest(root: str = ".") -> dict:
    """Summarize the project's purpose and structure."""
    from .packages import detect_project_stack, inspect_manifest
    from .filesystem import tree_project
    stack = detect_project_stack(root)
    readme = in_workspace(root) / "README.md"
    summary = ""
    if readme.exists():
        summary = readme.read_text(encoding="utf-8", errors="replace")[:2000]
    audit("read_project_manifest", {"root": root}, "summarized")
    return {"stack": stack, "tree": tree_project(root, max_depth=2, max_entries=60),
            "manifests": list(inspect_manifest(root).keys()), "readme_excerpt": summary}
```

load_task_context()

```
def load_task_context(task: str, max_files: int = 5) -> dict:
    """Load only the files and notes relevant to the current task."""
    notes = search_project_notes(task)
    from .code_intel import semantic_code_search
    files = [hit["file"] for hit in semantic_code_search(task, top_k=max_files)]
    audit("load_task_context", {"task": task[:200]}, f"{len(files)} files")
    return {"relevant_files": files, "related_notes": notes}
```

save_project_note()

```
def save_project_note(key: str, value: str) -> dict:
    """Save stable project-specific facts and decisions (secrets are scrubbed)."""
    notes = read_json(_NOTES, default={}) or {}
    notes[key] = {"value": _scrub(value), "updated": time.time()}
    write_json(_NOTES, notes)
    audit("save_project_note", {"key": key}, "saved")
    return {"key": key, "saved": True}
```

search_project_notes()

```
def search_project_notes(query: str) -> list[dict]:
    """Retrieve relevant notes from prior work."""
    notes = read_json(_NOTES, default={}) or {}
    q = query.lower()
    hits = [{"key": k, **v} for k, v in notes.items()
            if q in k.lower() or q in str(v.get("value", "")).lower()]
    audit("search_project_notes", {"query": query[:120]}, f"{len(hits)} notes")
    return hits[:20]
```

update_task_state()

```
def update_task_state(task: str, state: str, detail: str = "") -> dict:
    """Track task status, decisions, and blockers."""
    tasks = read_json(_TASKS, default={}) or {}
    tasks[task] = {"state": state, "detail": _scrub(detail), "updated": time.time()}
    write_json(_TASKS, tasks)
    audit("update_task_state", {"task": task[:200]}, state)
    return tasks[task]
```

record_decision()

```
def record_decision(decision: str, rationale: str) -> str:
    """Store a short architectural decision with rationale."""
    p = in_workspace(".agent") / "decisions.md"
    p.parent.mkdir(parents=True, exist_ok=True)
    entry = f"\n## {time.strftime('%Y-%m-%d %H:%M')}\n- **Decision:** {_scrub(decision)}\n- **Why:**
{_scrub(rationale)}\n"
    with p.open("a", encoding="utf-8") as fh:
        fh.write(entry)
    audit("record_decision", {"decision": decision[:200]}, "recorded")
    return str(p)
```

clear_task_memory()

```
def clear_task_memory(task: str) -> dict:
    """Remove temporary context after a task completes."""
    tasks = read_json(_TASKS, default={}) or {}
    removed = tasks.pop(task, None)
    write_json(_TASKS, tasks)
    audit("clear_task_memory", {"task": task[:200]}, "cleared" if removed else "not found")
    return {"task": task, "cleared": bool(removed)}
```

summarize_session()

```
def summarize_session(tasks_done: list[str], blockers: list[str],
                      next_step: str = "") -> str:
    """Produce a concise handoff summary for the next session."""
    done_lines = [f"- {t}" for t in tasks_done] or ["- (none)"]
    blocker_lines = [f"- {b}" for b in blockers] or ["- (none)"]
    lines = [f"# Session handoff — {time.strftime('%Y-%m-%d %H:%M')}", "",
             "## Completed", *done_lines, "",
             "## Blockers", *blocker_lines, "",
             f"## Next step\n{next_step or '(unspecified)'}"]
    p = in_workspace(".agent") / "session_summary.md"
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text("\n".join(lines), encoding="utf-8")
    audit("summarize_session", {"done": len(tasks_done)}, str(p))
    return "\n".join(lines)
```

2.17 Documentation and delivery

Tool	Purpose & constraints
`generate_readme`	Create/update setup and usage documentation.
`generate_api_docs`	Document endpoints and schemas.
`generate_architecture_doc`	Describe components and relationships.
`generate_setup_guide`	Provide reproducible environment setup.
`generate_changelog`	Summarize user-visible changes.
`generate_release_notes`	Prepare release notes from verified changes.
`generate_env_example`	Create a safe .env.example with placeholders.
`prepare_patch_summary`	Summarize files changed and why.
`prepare_delivery_report`	Report tests, builds, blockers, next steps.
`export_markdown`	Write Markdown documentation to a file.

generate_readme()

```
def generate_readme(path: str = ".") -> str:
    """Create or update project setup and usage documentation."""
    from .memory import read_project_manifest
    manifest = read_project_manifest(path)
    doc = _model_doc("Write a README (title, description, setup, usage, testing) for this project:",
                     extra=str(manifest)[:6000])
    export_markdown("README", doc, folder=".")
    audit("generate_readme", {"path": path}, "written")
    return doc
```

generate_api_docs()

```
def generate_api_docs(path: str = ".") -> str:
    """Document endpoints and schemas found in the project."""
    from .filesystem import search_files
    routes = search_files(r"(@app\.|@router\.|app\.(get|post|put|delete)|Router\()", path,
                          regex=True, max_results=60)
    if not routes:
        raise ToolError("No API route definitions found to document.")
    doc = _model_doc("Document these API routes (method, path, params, response):",
                     extra="\n".join(routes)[:6000])
    export_markdown("api-reference", doc)
    return doc
```

generate_architecture_doc()

```
def generate_architecture_doc(path: str = ".") -> str:
    """Describe components and their relationships."""
    from .code_intel import build_dependency_graph
    graph = build_dependency_graph(path)
    doc = _model_doc("Describe this project's architecture based on its module dependency graph:",
                     extra=str(dict(list(graph.items())[:80]))[:6000])
    export_markdown("architecture", doc)
    return doc
```

generate_setup_guide()

```
def generate_setup_guide(path: str = ".") -> str:
    """Provide reproducible environment setup steps."""
    from .packages import check_dependency_versions, detect_project_stack
    env = {**detect_project_stack(path), "versions": check_dependency_versions(path)}
    doc = _model_doc("Write a reproducible environment setup guide for this stack:",
                     extra=str(env)[:4000])
    export_markdown("setup-guide", doc)
    return doc
```

generate_changelog()

```
def generate_changelog(path: str = ".", limit: int = 50) -> str:
    """Summarize user-visible changes from git history."""
    from .core import run
    res = run(["git", "log", f"-{limit}", "--pretty=format:%h %ad %s", "--date=short"], cwd=path)
    if res["exit_code"] != 0 or not res["stdout"].strip():
        raise ToolError("No git history available for a changelog.")
    doc = _model_doc("Group these commits into a CHANGELOG with Added/Changed/Fixed sections:",
                     extra=res["stdout"][:8000])
    export_markdown("CHANGELOG", doc, folder=".")
    return doc
```

generate_release_notes()

```
def generate_release_notes(since_tag: str | None = None) -> str:
    """Prepare release notes from verified changes only."""
    from .core import run
    args = ["git", "log", "--pretty=format:%h %s"]
    if since_tag:
        args.append(f"{since_tag}..HEAD")
    res = run(args)
    if res["exit_code"] != 0:
        raise ToolError("Could not read git history for release notes.")
    doc = _model_doc("Write user-facing release notes (features, fixes, breaking changes) "
                     "from these commits. Do not invent changes:",
                     extra=res["stdout"][:8000])
    export_markdown("release-notes", doc)
    return doc
```

generate_env_example()

```
def generate_env_example(path: str = ".") -> str:
    """Create a safe .env.example with placeholder values, never real secrets."""
    from .security import inspect_secrets_presence
    keys: set[str] = set()
    env_file = in_workspace(path) / ".env"
    if env_file.exists():
        for line in env_file.read_text(encoding="utf-8", errors="replace").splitlines():
            m = re.match(r"^\s*([A-Z_][A-Z0-9_]*)\s*=", line)
            if m:
                keys.add(m.group(1))
    for finding in inspect_secrets_presence(path)[:20]:
        pass  # patterns only; we never read secret values
    if not keys:
        raise ToolError("No .env keys found to mirror.")
    content = "\n".join(f"{k}=changeme" for k in sorted(keys)) + "\n"
    p = in_workspace(path) / ".env.example"
    p.write_text(content, encoding="utf-8")
    audit("generate_env_example", {"keys": len(keys)}, str(p))
    return str(p)
```

prepare_patch_summary()

```
def prepare_patch_summary(path: str = ".") -> dict:
    """Summarize files changed and why."""
    from .core import run
    res = run(["git", "diff", "--name-status", "HEAD"], cwd=path)
    files = res["stdout"].splitlines()
    staged = run(["git", "diff", "--name-status", "--staged"], cwd=path)["stdout"].splitlines()
    return {"changed": files, "staged": staged,
            "note": "Use git_diff_summary() for a model-written behavioral summary."}
```

prepare_delivery_report()

```
def prepare_delivery_report(checks_run: list[dict], checks_skipped: list[str] | None = None,
                            blockers: list[str] | None = None,
                            next_step: str = "") -> dict:
    """Report tests, builds, blockers, and next steps; separate done from not done."""
    checks_skipped = checks_skipped or []
    blockers = blockers or []
    report = {
        "timestamp": time.strftime("%Y-%m-%d %H:%M"),
        "completed": [c for c in checks_run if c.get("passed")],
        "failed": [c for c in checks_run if not c.get("passed")],
        "not_run": checks_skipped,
        "blockers": blockers,
        "next_step": next_step,
    }
    report["overall"] = "PASS" if not report["failed"] and not blockers else "FAIL"
    audit("prepare_delivery_report", {"checks": len(checks_run)}, report["overall"])
    return report
```

export_markdown()

```
def export_markdown(name: str, content: str, folder: str = "docs") -> str:
    """Write Markdown documentation to a file with safe naming."""
    safe = re.sub(r"[^\w.-]+", "-", name).strip("-") or "doc"
    p = in_workspace(folder) / f"{safe}.md"
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    audit("export_markdown", {"name": name}, str(p))
    return str(p)
```

Minimal initial tool set

1. `list_files`
2. `read_file`
3. `search_files`
4. `git_status`
5. `git_diff`
6. `create_plan`
7. `apply_patch`
8. `run_command`
9. `run_unit_tests`
10. `prepare_delivery_report`

This set is enough to build a useful prototype while keeping the surface area manageable. Add more tools only when a real workflow requires them.

Final checklist

- The agent knows the approved project root.
- Tool arguments are schema-validated.
- Paths cannot escape the workspace unintentionally.
- Commands have timeouts and bounded output.
- Destructive or external actions require permission.
- Existing user changes are preserved.
- Tests and builds are run where appropriate.
- Failures and skipped checks are reported honestly.
- Secrets are redacted and not saved to memory.
- The final report lists changed files, checks run, and blockers.